

Sistema de Gestão de Escalas — PCCE

Apostila de Implantação

Do repositório no GitHub à produção no Cloudflare: infraestrutura,
variáveis, secrets e GitHub Actions, passo a passo

Setembro de 2026

Sumário

Como usar esta apostila

A regra que organiza o documento inteiro

Estrutura

Convenções

O que esta apostila já consertou no repositório

Parte 0 — Antes de começar

1. O que você vai construir

1.1 A arquitetura do deploy

1.2 Os ambientes

1.3 O que o sistema exige da infraestrutura

2. Contas, ferramentas e custos

2.1 Ferramentas na sua máquina

2.2 Autenticar o Wrangler

2.3 Custos, com honestidade

3. Decisões que você precisa tomar antes

3.1 Nomes dos recursos

3.2 Domínio

3.3 E-mail

3.4 Nível de assinatura digital

3.5 Integração com a planilha Google

4. Cofre: a disciplina que evita o desastre

4.1 A regra da Cloudflare

4.2 Os segredos *load-bearing*

4.3 Segredos que precisam bater com outro sistema

4.4 O que não precisa guardar

4.5 Como gerar

Parte I — Infraestrutura Cloudflare

5. Conta e Account ID

6. Banco de dados D1

6.1 Criar os bancos

6.2 Colar os IDs no `wrangler.toml`

6.3 O que NÃO fazer

7. Armazenamento R2

7.1 Criar os buckets

7.2 A regra inegociável: nenhum bucket é público

7.3 Retenção e imutabilidade (bucket de backup)

8. O projeto Pages

8.1 Criar como Direct Upload

8.2 Por que não conectar ao GitHub

8.3 Compatibilidade e bindings

9. Tokens de API da Cloudflare

9.1 Token de deploy (`CLOUDFLARE_API_TOKEN`)9.2 Token de backup (`CLOUDFLARE_BACKUP_API_TOKEN`)

10. Domínio e DNS

10.1 Adicionar o domínio ao projeto

10.2 Fixar a origem na aplicação

10.3 A URL de preview

Parte II — Variáveis e secrets do Cloudflare

11. Como configurar

11.1 Os três lugares onde uma variável pode viver

11.2 Production e Preview são dois conjuntos separados

11.3 Runtime, não build

11.4 Uma variável só passa a valer no próximo deploy

12. Catálogo A — o mínimo para o sistema funcionar

`SYNC_TOKEN` 🔒E-mail funcionando (`EMAIL` binding ou `RESEND_API_KEY`)`APP_ORIGIN`

13. Catálogo B — os segredos que protegem em silêncio

`PASSWORD_PEPPER` 🔒 *load-bearing*`CPF_ENCRYPTION_KEY` 🔒 *load-bearing*`CPF_INDEX_KEY` 🔒 *load-bearing*`AUDIT_CHAIN_KEY` 🔒 *load-bearing*`AUDIT_IP_ENCRYPTION_KEY` 🔒`RATE_LIMIT_IP_SALT` 🔒

14. Catálogo C — e-mail

Binding `EMAIL` (Cloudflare Email Sending)`CLOUDFLARE_API_TOKEN` + `CLOUDFLARE_ACCOUNT_ID``RESEND_API_KEY` 🔒 e `RESEND_FROM_EMAIL`

Como testar antes do go-live

15. Catálogo D — contas de bootstrap

`SUPER_ADMIN_LOGIN` / `SUPER_ADMIN_SENHA` 🔒 / `SUPER_ADMIN_EMAIL``ADMIN_GERAL_LOGIN` / `ADMIN_GERAL_SENHA` 🔒 / `ADMIN_GERAL_EMAIL`

16. Catálogo E — webhooks e operações destrutivas

`RESET_TOKEN` 🔒`WEBHOOK_REPLAY_ENFORCE``WEBHOOK_ALLOW_PAPEL_CHANGES`

17. Catálogo F — assinatura digital

17.1 Trust store ICP-Brasil

`ICP_BRASIL_TRUST_STORE_REQUIRED`

17.2 Carimbo de tempo (TSA)

`TSA_URL``TSA_USERNAME` / `TSA_PASSWORD` 🔒`EXIGIR_TSA_QUALIFICADA`

17.3 Selo institucional (assinatura avançada)

`SELO_INSTITUCIONAL_PEM` 🗝️ *load-bearing*

17.4 Ajustes avançados (deixe como estão)

`EMBED_PADES_LT_DSS`

`PA_AD_RB_HASH_HEX`

18. Catálogo G — observabilidade e ajuste fino

`SENTRY_DSN` / `SENTRY_ENVIRONMENT` / `SENTRY_TRACES_SAMPLE_RATE`

`PUBLIC_SENTRY_DSN` / `PUBLIC_SENTRY_ENVIRONMENT`

`HEALTH_DETAIL_TOKEN` 🗝️

`SESSION_CACHE_TTL_SECONDS`

19. Catálogo H — integração com a planilha Google

`GISE_BASE_EQUIPE_WEBHOOK_URL`

`GISE_BASE_EQUIPE_SECRET` 🗝️

Passo a passo da planilha

20. Tabela-resumo de todas as variáveis

21. Conferindo o que realmente pegou

Parte III — GitHub Actions

22. Os quatro workflows

23. Onde ficam os segredos do GitHub

24. Environments: o portão de produção

25. `deploy.yml` — o pipeline principal

25.1 Secrets necessários

25.2 O que o pipeline faz, na ordem

25.3 Proteção de branch

25.4 Duas decisões do workflow que você não deve "otimizar"

26. `cleanup-retencao.yml` — a limpeza da LGPD

26.1 Secrets

26.2 Por que ele pode dar 401 mesmo com os secrets certos

26.3 O failsafe: monitore o atraso

26.4 O número que você monitora depois

27. `backup-d1.yml` — o backup cifrado

27.1 O que ele faz

27.2 Setup, na ordem

27.3 Teste de restauração (não pule)

27.4 O outro mecanismo: Time Travel

28. `update-icp-brasil-trust-store.yml` — a cadeia da ITI

28.1 O que ele faz

28.2 Permissões

28.3 Notificação por e-mail (opcional)

29. Dependabot

30. Tabela-resumo dos secrets do GitHub

Parte IV — Primeiro deploy e go-live

31. Preparar o repositório

- 31.1 Obter o código
- 31.2 Conferir o que precisa ser seu
- 31.3 Validar localmente (recomendado)

32. Migrações do banco

- 32.1 As três formas
- 32.2 O caso especial: banco que já está migrado
- 32.3 O que algumas migrações SEMEIAM
- 32.4 Ordem segura

33. O primeiro deploy

- 33.1 Pelo GitHub Actions (recomendado)
- 33.2 Manual (fallback)
- 33.3 Se o deploy falhar

34. Smoke tests pós-deploy

35. As primeiras contas

- 35.1 Entrar como Super Admin
- 35.2 A ordem de povoamento
- 35.3 Quem pode o quê
- 35.4 Desligue o bootstrap do Admin Geral

36. Configuração inicial do sistema

- 36.1 Política de assinatura (`/conf-ass` , Super Admin)
- 36.2 Valores de custo (`/config-custos` , Super Admin)
- 36.3 Modelos de reconhecimento facial

37. Go-live: o reset de senhas

- 37.1 Pré-requisitos absolutos
- 37.2 O comando
- 37.3 Contas em formato de hash antigo

38. Checklist de go-live

Parte V — Operação

39. A rotina

- Diária (automática, você só confere)
- Semanal
- Mensal
- Trimestral

40. Rollback

41. Rotação de segredos

- 41.1 O que NUNCA se rotaciona
- 41.2 O que se rotaciona, com cuidado
- 41.3 O que se rotaciona sem drama

42. Solução de problemas por sintoma

- Ninguém consegue logar
- Assinatura falhando
- Automação parada
- Ambiente errado

43. Palavra final

Apêndices

Apêndice A — Modelo de inventário de variáveis

Apêndice B — Comandos de referência

Apêndice C — Permissões dos tokens de API

Apêndice D — Gerar o hash da senha de bootstrap

Apêndice E — Glossário de implantação

Como usar esta apostila

Esta apostila leva você **do zero até o sistema no ar**: criar a infraestrutura na Cloudflare, configurar cada variável de ambiente e cada segredo, ligar o GitHub Actions, rodar o primeiro deploy e fazer o go-live.

Ela é escrita para quem **nunca implantou este sistema** e não necessariamente conhece Cloudflare. Todo passo tem: o comando exato, onde clicar no painel, o que a coisa faz e — o mais importante — **o que acontece se você não fizer**.

A regra que organiza o documento inteiro

Este sistema tem uma característica que muda a forma de implantá-lo: **várias proteções desligam em silêncio**. Se você esquecer o `CPF_ENCRYPTION_KEY`, nada quebra — o sistema sobe, aceita cadastro, funciona, e grava o CPF de sete mil servidores **em texto puro** no banco. Se você esquecer o `AUDIT_CHAIN_KEY`, a trilha de auditoria continua dizendo "cadeia íntegra", só que agora qualquer um com acesso de escrita ao banco pode forjá-la inteira.

Por isso, cada variável desta apostila vem com três informações:

Campo	O que responde
Função	O que a variável liga
Se faltar	O comportamento exato do sistema sem ela — quebra, degrada em silêncio ou é inócuo
Pode trocar depois?	Se é rotacionável ou <i>load-bearing</i> (trocar destrói dado já gravado)

Leia a coluna "Se faltar" mesmo das variáveis que você decidir não usar. Ela é o inventário de riscos da instalação.

Estrutura

- **Parte 0 — Antes de começar.** O que você vai construir, contas, ferramentas, decisões e a disciplina de cofre.
- **Parte I — Infraestrutura Cloudflare.** D1, R2, projeto Pages, tokens de API, domínio.
- **Parte II — Variáveis e secrets do Cloudflare.** O catálogo completo, agrupado por assunto, com a tabela-resumo no fim.
- **Parte III — GitHub Actions.** Os quatro workflows, os secrets de cada um, environments e proteção de branch.
- **Parte IV — Primeiro deploy e go-live.** Migrações, deploy, smoke tests, criação do Super Admin, reset de senhas, configuração inicial do sistema.
- **Parte V — Operação.** Rotina, backup, rollback, rotação de segredos e solução de problemas por sintoma.
- **Apêndices.** Tabelas de consulta rápida: variáveis, secrets, comandos, permissões de token e um script pronto para gerar hash de senha.

Convenções

- Blocos cinza são comandos de terminal. Rode-os na raiz do repositório, salvo indicação em contrário.
- Onde aparece `<algo>`, substitua pelo seu valor (sem os sinais de menor/maior).

-  marca armadilha conhecida — coisa que já deu errado.
-  marca o jeito correto.
-  marca dica de operação.
-  marca segredo que precisa ir para o cofre.

O que esta apostila já consertou no repositório

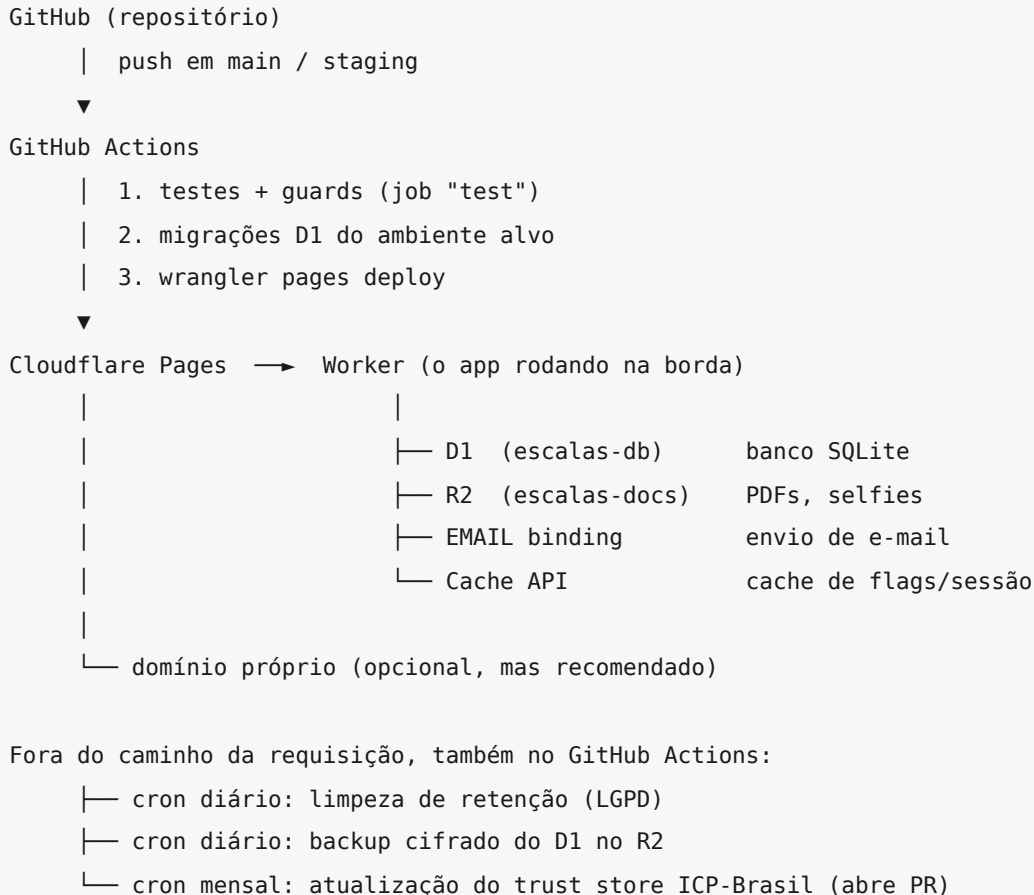
Escrever este documento do começo ao fim expôs pontos em que a documentação afirmava um estado que o código já não tinha. Eles foram **corrigidos no mesmo ciclo**, e ficam registrados aqui porque explicam decisões que você vai encontrar:

1. **scripts/hash-senha.mjs** passou a existir. O `DEPLOY.md` mandava gerar o hash da senha de bootstrap com um `scripts/hash-password.ts` que nunca foi commitado. O script agora existe, tem teste que executa o arquivo de verdade e confere a saída contra o `verificarSenha` do login, e o runbook aponta para ele. Ver apêndice D.
2. **A duração da sessão é 1 hora de inatividade**, não 8 — o `README.md` dizia 8 em dois lugares e foi corrigido.
3. **O salt do rate-limit era lido da fonte errada.** `RATE_LIMIT_IP_SALT` só era lido de `process.env`, que no Pages não é a fonte canônica, enquanto o `/api/health` conferia sua presença em `platform.env`. Ou funcionava por acidente da data de compatibilidade, ou a proteção estava desligada com o failsafe verde. As duas metades agora leem a mesma fonte.
4. **O tipo `Env (src/app.d.ts)` ganhou as dez variáveis que faltavam**, entre elas quatro segredos de proteção. Os casts `as Record<string, unknown>` que as mantinham fora dos tipos saíram — um typo em `WEBHOOK_REPLAY_ENFORCE` compilava e desligava a proteção em silêncio.

Parte 0 — Antes de começar

1. O que você vai construir

1.1 A arquitetura do deploy



Três coisas nessa figura costumam surpreender quem chega:

- **O deploy não é feito pela Cloudflare.** Quem builda e publica é o GitHub Actions, usando o Wrangler. O projeto Pages é do tipo *Direct Upload*, não conectado ao Git (o capítulo 8 explica por quê).
- **As migrações de banco rodam no CI, antes do deploy.** Se a migração falhar, o deploy não acontece.
- **Não existe cron dentro do Cloudflare Pages.** Toda tarefa periódica é um workflow agendado no GitHub que chama um endpoint autenticado do próprio sistema. Se você desligar o Actions, três rotinas param — e duas delas param **em silêncio**.

1.2 Os ambientes

Ambiente	Branch	Onde roda	Banco	Bucket
Produção	main	Pages (produção)	escalas-db	escalas-docs
Staging	staging	Pages (preview)	escalas-db-staging	escalas-docs-staging
Local	—	Wrangler na sua máquina	SQLite em .wrangler/	simulado

Staging é opcional para subir, e **obrigatório** para operar com segurança: é onde uma migração nova é testada antes de tocar o banco que guarda documento assinado.

⚠ O `wrangler.toml` traz um aviso importante: enquanto o `database_id` do staging for um placeholder, os deploys de preview **falham ao ligar o binding**. Isso é proposital (*fail-safe*) — é melhor o preview quebrar do que escrever no banco de produção.

1.3 O que o sistema exige da infraestrutura

Recurso	Para quê	Obrigatório?
Conta Cloudflare (Pages + D1 + R2)	Rodar a aplicação, o banco e os arquivos	Sim
Repositório GitHub	Código e automações	Sim
Provedor de e-mail	2FA, primeiro acesso, reset de senha	Sim — sem e-mail ninguém loga
Domínio próprio	URL estável e RP ID do WebAuthn	Recomendado (obrigatório com passkey)
Conta Sentry	Captura de erros 5xx	Recomendado
ACT de carimbo de tempo (ICP-Brasil)	Peso probatório pleno da assinatura qualificada	Só para assinatura qualificada com valor ICP pleno
Planilha Google + Apps Script	Sincronizar cadastro de servidores e unidades	Opcional

2. Contas, ferramentas e custos

2.1 Ferramentas na sua máquina

```
# Node.js 22 ou superior (o CI usa 22)
node --version

# Wrangler — a CLI da Cloudflare
npm install -g wrangler@latest
wrangler --version

# OpenSSL — para gerar todos os segredos
openssl version

# age — só se você for configurar o backup cifrado (Parte III)
```

```
# Ubuntu/Debian: sudo apt-get install age
# macOS:         brew install age
# Windows:       https://github.com/FiloSottile/age/releases
age --version
```

No Windows, prefira o **WSL** ou o **Git Bash**: os comandos desta apostila usam `openssl` e sintaxe de shell POSIX.

2.2 Autenticar o Wrangler

```
wrangler login      # abre o navegador para autorizar
wrangler whoami     # confirma a conta e mostra o Account ID
```

💡 Guarde o **Account ID** que o `whoami` imprime — ele é usado em vários passos e vira o secret `CLOUDFLARE_ACCOUNT_ID` no GitHub.

2.3 Custos, com honestidade

Serviço	Plano	Observação
Cloudflare Pages	Gratuito atende	Limite de builds não se aplica: quem builda é o GitHub
Cloudflare D1	Gratuito tem cota de leitura/escrita e 5 GB	Uma corporação de milhares de servidores cabe; monitore
Cloudflare R2	Cobrado por armazenamento; sem taxa de saída	PDFs assinados e selfies acumulam — planeje a retenção
Cloudflare Email Sending	Incluído	Exige domínio na conta (ver capítulo 14)
Resend	Plano gratuito limitado	Fallback do e-mail
Sentry	Plano gratuito atende	
ACT ICP-Brasil (carimbo)	Pago	Só se você exigir carimbo qualificado
GitHub Actions	Gratuito em repositório público; cota em privado	O CI completo leva ~16 min por push

⚠️ Não use o plano gratuito do Resend com um remetente próprio: sem domínio verificado, o remetente **tem** de ser `onboarding@resend.dev`, e e-mail institucional saindo desse endereço costuma cair em spam. Para produção de verdade, verifique o domínio.

3. Decisões que você precisa tomar antes

Anote as respostas antes de criar qualquer recurso — várias delas são difíceis de mudar depois.

3.1 Nomes dos recursos

O repositório já vem com nomes fixados no `wrangler.toml` e nos workflows:

Recurso	Nome esperado	Onde está escrito
Projeto Pages	<code>escalas</code>	<code>deploy.yml (--project-name=escalas)</code>
D1 de produção	<code>escalas-db</code>	<code>wrangler.toml</code> , <code>scripts/migrate.ts</code> , <code>backup-d1.yml</code>
D1 de staging	<code>escalas-db-staging</code>	<code>wrangler.toml</code> , <code>scripts/migrate.ts</code>
R2 de produção	<code>escalas-docs</code>	<code>wrangler.toml</code>
R2 de staging	<code>escalas-docs-staging</code>	<code>wrangler.toml</code>
R2 de backup	<code>escalas-backups</code>	<code>backup-d1.yml</code>

✓ Use **exatamente** esses nomes. Trocar um deles obriga a editar `wrangler.toml`, `scripts/migrate.ts` e dois workflows — e o `migrate.ts` tem o nome do banco embutido na lógica de confirmação de produção.

3.2 Domínio

Três opções, em ordem de preferência:

1. **Domínio próprio** (`escalas.suainstituicao.gov.br`) — o certo. É a URL que vai nos e-mails e o **RP ID** das chaves de assinatura.
2. `*.pages.dev` **provisório, com plano de migrar** — aceitável para piloto, desde que **a chave de assinatura (passkey) esteja desligada**.
3. `*.pages.dev` definitivo — funciona, mas é URL de teste num sistema que emite documento oficial.

⚠ **A decisão do domínio é quase irreversível se a passkey estiver ligada.** A credencial WebAuthn fica presa ao domínio em que foi criada. Registrar em `escalas.pages.dev` e depois migrar para o domínio próprio **invalida todas as chaves** — todo servidor precisa recadastrar a dele. Decida o domínio **antes** de ligar `exigir_passkey_assinatura`.

3.3 E-mail

Caminho	Quando escolher
Binding EMAIL (Cloudflare Email Sending)	Você controla o domínio <code>escalaspcce.com.br</code> ou vai ajustar a constante do remetente
Resend	Caminho mais simples para subir; funciona como primário ou fallback
Os dois	Recomendado: um assume quando o outro falha ou estoura a cota

⚠ **Detalhe que pega todo mundo:** o remetente do caminho Cloudflare é **fixo no código** — `sistema@nao-responda.escalaspcce.com.br`, em `src/lib/server/email.ts`. Se o seu domínio for outro, o envio pela Cloudflare falha e cai no Resend. Para usar o binding **EMAIL** com domínio próprio, altere a constante `CF_FROM` no mesmo PR em que configurar o domínio.

3.4 Nível de assinatura digital

Cenário	O que configurar
Só assinatura avançada (2FA + selfie + GPS + selo)	<code>SELO_INSTITUCIONAL_PEM</code> ; sem TSA obrigatória
Assinatura qualificada (e-CPF A3), aceitando carimbo não-ICP	Trust store populado + <code>ICP_BRASIL_TRUST_STORE_REQUIRED=1</code>
Assinatura qualificada com carimbo ICP pleno	O acima + <code>TSA_URL</code> de uma ACT credenciada + <code>EXIGIR_TSA_QUALIFICADA=1</code>

⚠ A ordem importa e há uma armadilha explosiva aqui: ligar `EXIGIR_TSA_QUALIFICADA=1` **sem** trocar a `TSA_URL` faz o sistema **rejeitar 100% das assinaturas qualificadas com HTTP 422**. O capítulo 18 detalha.

3.5 Integração com a planilha Google

Opcional. Se o cadastro de servidores vem de uma planilha, você vai precisar de dois pares de segredos (`SYNC_TOKEN / RESET_TOKEN` e `GISE_BASE_EQUIPE_*`) e de publicar um Apps Script. Capítulo 19.

4. Cofre: a disciplina que evita o desastre

4.1 A regra da Cloudflare

A Cloudflare **não mostra o valor de um secret depois de salvo**. O painel exibe "Valor criptografado" e ponto. Se você gerar um segredo, colar no painel e não guardar em outro lugar, ele está perdido para sempre.

Para a maioria das variáveis isso é só inconveniente — você gera outra. Para quatro delas, **é catastrófico**: perder invalida dado já gravado.

4.2 Os segredos *load-bearing*

🔑 Gere uma vez, guarde em cofre, nunca rotacione:

Segredo	O que se perde ao trocar
<code>PASSWORD_PEPPER</code>	Todas as senhas em formato <code>pbkdf2v3</code> param de verificar — a base inteira precisa redefinir senha
<code>CPF_ENCRYPTION_KEY</code>	Os CPFs cifrados viram lixo indecifrável
<code>CPF_INDEX_KEY</code>	O índice cego para de casar; o login por certificado não acha o titular
<code>AUDIT_CHAIN_KEY</code>	As linhas antigas da trilha ficam inverificáveis
<code>SELO_INSTITUCIONAL_PEM</code>	A identidade do selo muda; você nunca regenera a mesma chave

4.3 Segredos que precisam bater com outro sistema

🔑 Guarde porque você vai precisar **reusar o valor exato** em outro lugar:

Segredo	Onde mais ele aparece
SYNC_TOKEN	Apps Script da planilha e secret do GitHub Actions (cron de limpeza)
RESET_TOKEN	Apps Script da planilha
GISE_BASE_EQUIPE_SECRET	Script Properties da planilha
SUPER_ADMIN_SENHA / ADMIN_GERAL_SENHA	É credencial de acesso — sem ela você não entra
CLOUDFLARE_API_TOKEN	GitHub Actions
BACKUP_AGE_PUBLIC_KEY (e a privada)	A privada é a única forma de ler os backups

4.4 O que não precisa guardar

CLOUDFLARE_ACCOUNT_ID (está no painel), SENTRY_DSN (idem), APP_ORIGIN, TSA_URL, RESEND_FROM_EMAIL, os *_LOGIN e todas as flags (WEBHOOK_REPLAY_ENFORCE, EXIGIR_TSA_QUALIFICADA, etc.). São recuperáveis ou públicos.

4.5 Como gerar

Todo segredo aleatório deste sistema é gerado do mesmo jeito:

```
openssl rand -hex 32
```

São 32 bytes → 64 caracteres hexadecimais → 256 bits. Gere **um valor diferente para cada variável**. Reutilizar o mesmo valor em duas anula o motivo de serem duas (o caso mais grave: RESET_TOKEN igual ao SYNC_TOKEN derruba a separação que impede que vazar o token de webhook baste para apagar o banco).

💡 Um jeito prático de gerar tudo de uma vez e já colar no cofre:

```
for v in SYNC_TOKEN RESET_TOKEN PASSWORD_PEPPER CPF_ENCRYPTION_KEY CPF_INDEX_KEY \
    AUDIT_CHAIN_KEY AUDIT_IP_ENCRYPTION_KEY RATE_LIMIT_IP_SALT \
    HEALTH_DETAIL_TOKEN GISE_BASE_EQUIPE_SECRET; do
    echo "$v=$(openssl rand -hex 32)"
done
```

⚠ Esse comando imprime segredos no terminal, que fica no histórico do shell. Rode com o histórico desligado (unset HISTFILE na sessão), cole no cofre e feche o terminal.

Parte I — Infraestrutura Cloudflare

Nesta parte você cria os recursos. Cada passo tem o comando do Wrangler (que é estável e verificável) e o caminho equivalente no painel. Os nomes de menu da Cloudflare mudam de tempos em tempos — **prefira o comando**; o painel serve para conferir o resultado.

5. Conta e Account ID

1. Crie (ou use) uma conta em <<https://dash.cloudflare.com>>.
2. Confirme que a conta tem **Workers & Pages, D1** e **R2** habilitados. O R2 exige adicionar uma forma de pagamento mesmo dentro da cota gratuita.
3. Pegue o Account ID:

```
wrangler whoami
```

Anote o valor. Ele vira o secret `CLOUDFLARE_ACCOUNT_ID` no GitHub e é usado no caminho de e-mail via API.

6. Banco de dados D1

6.1 Criar os bancos

```
# Produção
wrangler d1 create escalas-db

# Staging (crie mesmo que vá usar depois – o preview quebra sem ele)
wrangler d1 create escalas-db-staging
```

Cada comando imprime um bloco parecido com:

```
[[d1_databases]]
binding = "escalas_db"
database_name = "escalas-db"
database_id = "dc86ec72-7ed4-4e8c-9d29-67a4e509ea49"
```

6.2 Colar os IDs no `wrangler.toml`

Abra `wrangler.toml` e substitua os dois `database_id`:

```
[[d1_databases]]
binding = "escalas_db"
database_name = "escalas-db"
database_id = "<ID DO SEU escalas-db>"      # ← produção
migrations_dir = "migrations"
```

```
# ... mais abaixo, na seção de preview:

[[env.preview.dl_databases]]
binding = "escalas_db"
database_name = "escalas-db-staging"
database_id = "<ID DO SEU escalas-db-staging>" # ← staging
migrations_dir = "migrations"
```

⚠ **Este é o passo esquecido com mais frequência.** O repositório vem com os IDs da instalação original. Se você não trocar, o deploy tenta ligar a um banco que não é seu e falha — ou, pior, num fork mal configurado, aponta para um banco que existe e não deveria ser tocado.

Se faltar: o binding falha e toda rota que consulta o banco devolve 500. O `/api/health` responde `down`.

✅ **Commite essa alteração:** o `database_id` **não é segredo** (é um identificador; o acesso vem do token de API).

6.3 O que NÃO fazer

⚠ Não rode `wrangler dl migrations apply` à mão. O projeto tem um runner próprio (`scripts/migrate.ts`) que usa a tabela de controle `_migrations_aplicadas`. O comando nativo do Wrangler grava numa tabela **diferente** (`dl_migrations`), e misturar os dois já produziu um bug conhecido: a migração `0038` reexecuta, refaz uma tabela sem as colunas da `0060`, e o erro de coluna duplicada marca a `0060` inteira como aplicada — as colunas do relatório extra nunca voltam. O sintoma é assinatura preparando com 200 e finalizando com 500.

7. Armazenamento R2

7.1 Criar os buckets

```
wrangler r2 bucket create escalas-docs          # produção
wrangler r2 bucket create escalas-docs-staging  # staging
wrangler r2 bucket create escalas-backups       # backup do banco (Parte III)
```

7.2 A regra inegociável: nenhum bucket é público

Os três guardam dado pessoal: PDFs assinados com manifesto forense (CPF, IP, GPS), **selfies** de assinatura e, no caso do `escalas-backups`, o dump inteiro do banco.

No painel (R2 → bucket → Settings), confirme que **Public access** está desativado e que **não** há domínio público (`r2.dev`) habilitado.

Se faltar essa conferência: qualquer pessoa com a URL do objeto baixa a selfie e o PDF forense de um servidor público, sem autenticação. O download legítimo do sistema passa por rota autenticada que lê a chave do banco — o

bucket nunca precisa ser público.

7.3 Retenção e imutabilidade (bucket de backup)

No `escalas-backups`, configure no painel:

- **Lifecycle:** apagar objetos de `d1/diario/` após **90 dias** e de `d1/mensal/` após **12 meses**.
- **Retention lock:** impede deleção dentro da janela mesmo que o token seja comprometido.

Se faltar: o bucket cresce indefinidamente (custo), e um invasor com o token de backup pode apagar os backups junto com o ataque.

8. O projeto Pages

8.1 Criar como Direct Upload

```
wrangler pages project create escalas --production-branch=main
```

8.2 Por que não conectar ao GitHub

A Cloudflare oferece conectar o projeto Pages diretamente ao repositório. **Não faça isso neste projeto.** Motivos:

1. **Deploy duplo.** O GitHub Actions já builda e publica. Com a conexão Git, cada push dispararia dois deploys — o do Actions e o da Cloudflare — competindo pela mesma URL.
2. **O deploy da Cloudflare pularia os testes e os guards.** O `deploy.yml` só publica depois de lint, type-check, testes unitários, seis guards de padrão e a suíte E2E. A build da Cloudflare publica o que estiver lá.
3. **As migrações não rodariam.** Elas acontecem no job do Actions, imediatamente antes do `pages deploy`.

✓ Direct Upload + GitHub Actions é a topologia que este repositório assume, e é a que o `deploy.yml` implementa.

8.3 Compatibilidade e bindings

O `wrangler.toml` já declara o que o Worker precisa:

```
compatibility_date = "2026-04-01"
compatibility_flags = ["nodejs_compat"]
pages_build_output_dir = ".svelte-kit/cloudflare"
```

`nodejs_compat` é **obrigatório**: o código de assinatura digital usa APIs de Node (`node:crypto`, `node-forge`).

Se faltar: o deploy até acontece, e o Worker quebra em runtime na primeira operação de assinatura, com erro de módulo não encontrado.

💡 Depois do primeiro deploy, confira no painel (Workers & Pages → `escalas` → Settings) que a data de compatibilidade e a flag `nodejs_compat` aparecem, e que os bindings D1/R2 estão lá — nos dois escopos, Production e Preview. Se o painel não tiver refletido o `wrangler.toml`, configure as bindings manualmente; o próprio `wrangler.toml` do projeto avisa sobre essa possibilidade na seção de preview.

9. Tokens de API da Cloudflare

Você vai criar **dois** tokens, com escopos diferentes. Painel: **My Profile** → **API Tokens** → **Create Token** → **Create Custom Token**.

9.1 Token de deploy (`CLOUDFLARE_API_TOKEN`)

Usado pelo GitHub Actions para migrar o banco e publicar o site.

Permissão	Nível	Para quê
Cloudflare Pages — Edit	Conta	<code>wrangler pages deploy</code>
D1 — Edit	Conta	As migrações rodam no CI antes do deploy

⚠️ **A permissão de D1 é a esquecida.** Um token só com Pages publica o site e falha no passo anterior, na migração — e o log mostra um 403 genérico da API. O `DEPLOY.md` registra isso explicitamente.

💡 Se o Wrangler reclamar de não conseguir resolver a conta, acrescente **Account Settings — Read**. Passar o `CLOUDFLARE_ACCOUNT_ID` (que o workflow já faz) normalmente dispensa essa permissão.

🔑 Copie o token **na hora**: a Cloudflare mostra o valor uma única vez.

9.2 Token de backup (`CLOUDFLARE_BACKUP_API_TOKEN`)

Usado só pelo workflow de backup. **Não reutilize o token de deploy.**

Permissão	Nível	Para quê
D1 — Read	Conta	<code>wrangler d1 export</code>
Workers R2 Storage — Edit	Conta, restrito ao bucket <code>escalas-backups</code>	Gravar o dump cifrado

O princípio: o token que roda todo dia num runner público consegue **ler** o banco e **escrever num único bucket** — não consegue publicar código nem apagar dados.

💡 Se o `d1 export` devolver 403 com o token só de leitura, promova a permissão de D1 para **Edit** nesse token: a API de export da Cloudflare já exigiu escrita em algumas versões. Verifique com um disparo manual do workflow antes de confiar no cron.

10. Domínio e DNS

10.1 Adicionar o domínio ao projeto

Painel: **Workers & Pages** → **escalas** → **Custom domains** → **Set up a custom domain**.

- Se o domínio já está na Cloudflare, o registro DNS é criado sozinho.
- Se está em outro provedor, a Cloudflare mostra o **CNAME** a cadastrar lá.

Aguarde o certificado TLS ficar ativo (costuma levar poucos minutos).

10.2 Fixar a origem na aplicação

Depois que o domínio responder, defina a variável **APP_ORIGIN** (capítulo 12) com **exatamente** a origem canônica, com **https://** e **sem barra final**:

```
APP_ORIGIN=https://escalas.suainstituicao.gov.br
```

Função: é a origem usada nos links de e-mail (redefinição de senha, primeiro acesso) e o **RP ID** do WebAuthn.

Se faltar: o sistema cai na origem da própria requisição, derivada do header **Host**. Funciona, mas: (a) você perde a defesa em camadas contra *host-header injection*; (b) com a chave de assinatura ligada, a credencial fica presa ao host em que foi criada — e migrar de domínio depois invalida todas.

⚠ Se você já tem usuários com passkey registrada, **não mude APP_ORIGIN**. Isso equivale a exigir recadastro de chave de toda a corporação.

10.3 A URL de preview

O ambiente de staging responde numa URL de preview do Pages. Ela tem **secrets próprios** (escopo Preview) — inclusive um **SYNC_TOKEN** diferente, se você quiser. Guarde essa URL: ela é útil nos testes e é a causa nº 2 de erro 401 no cron de limpeza (apontar o **APP_BASE_URL** para o preview em vez da produção).

Parte II — Variáveis e secrets do Cloudflare

Esta é a parte mais longa da apostila, e é de propósito: **quase todo problema grave de implantação deste sistema é uma variável faltando**. Cada item traz função, consequência da ausência e se pode ser trocado depois.

11. Como configurar

11.1 Os três lugares onde uma variável pode viver

Lugar	O que é	Quando usar
<code>wrangler.toml</code> → <code>[vars]</code>	Valor público , versionado no Git	Só configuração não sensível. Hoje só <code>TSA_URL</code>
Painel Pages → Environment variables	Valor no ambiente, com opção Encrypt	O caminho normal para tudo, secreto ou não
<code>wrangler pages secret put</code>	O mesmo, pela linha de comando	Automação, ou valor grande demais para colar (o selo institucional)

```
# Equivalente em CLI ao painel (escopo de produção)
wrangler pages secret put PASSWORD_PEPPER --project-name=escalas

# Escopo de preview (staging)
wrangler pages secret put PASSWORD_PEPPER --project-name=escalas --env=preview

# Conferir o que existe (mostra os NOMES, nunca os valores)
wrangler pages secret list --project-name=escalas
```

 **Marque "Encrypt" para todo segredo no painel.** Uma variável não criptografada fica legível para qualquer pessoa com acesso ao painel — e, uma vez salva sem criptografia, o valor já foi exposto: gere outro.

11.2 Production e Preview são dois conjuntos separados

O Pages tem dois escopos de variáveis. **Uma variável definida em Production não existe no Preview.** Consequências práticas:

- O staging precisa da sua própria cópia de tudo que for exercitar.
- Para o staging testar o caminho `pbkdf2v3`, defina um `PASSWORD_PEPPER` no escopo Preview — pode ser um valor de teste, diferente do de produção (os bancos são isolados).
-  Um `SYNC_TOKEN` diferente no Preview é a razão de o cron de limpeza dar 401 quando alguém aponta o `APP_BASE_URL` para a URL de preview.

11.3 Runtime, não build

Todas as variáveis deste sistema são lidas **em runtime**, dentro do Worker — inclusive as `PUBLIC_*`, que usam `$env/dynamic/public`. Isso tem duas consequências boas:

- Você **não precisa rebuildar** para trocar o valor de uma variável. Basta salvar e fazer um novo deploy (ou aguardar o próximo) para o Worker recarregar o ambiente.
- Nenhum segredo entra no bundle JavaScript.

⚠ No painel do Pages existe a distinção entre variáveis de **build** e de **runtime**. Cadastre em runtime ("Variáveis e segredos" do projeto). O `scripts/README.md` registra esse erro: a integração da planilha reclamava "URL ou secret ausente" com os valores corretos — cadastrados só no lugar errado.

11.4 Uma variável só passa a valer no próximo deploy

Salvar no painel não afeta o deployment que já está no ar. Faça um novo deploy (push, ou **Retry deployment** no painel) depois de alterar variáveis.

12. Catálogo A — o mínimo para o sistema funcionar

Sem estas três, o sistema não é utilizável.

`SYNC_TOKEN` 🔑

- **Função:** *bearer token* dos webhooks de sincronização (`/api/webhook/sync-policiais`, `/api/webhook/sync-unidades`), camada base do endpoint destrutivo e credencial do cron de limpeza de retenção.
- **Gerar:** `openssl rand -hex 32`.
- **Regra de tamanho:** o sistema **recusa** tokens com menos de **32 caracteres** (`SYNC_TOKEN_MIN_LEN`), respondendo 401 — é *fail-closed* contra segredo fraco em produção.
- **Se faltar:** os webhooks respondem 401. Na prática: a sincronização da planilha para de funcionar **e a limpeza de retenção da LGPD para de rodar**, silenciosamente, deixando sessões, tokens e nonces se acumularem.
- **Pode trocar depois?** Sim — mas troque nos **três** lugares ao mesmo tempo: Cloudflare (Production), secret do GitHub Actions e Script Properties da planilha.

E-mail funcionando (`EMAIL` binding ou `RESEND_API_KEY`)

- **Função:** enviar código 2FA, convite de primeiro acesso e link de redefinição de senha.
- **Se faltar:** **ninguém entra no sistema**. O 2FA é *fail-closed* — conta sem e-mail entregue não recebe sessão. O primeiro acesso também trava. É a falha mais grave possível nesta lista, e ela não aparece em nenhum teste automatizado: o deploy fica verde e o login não completa.
- Detalhes no capítulo 14.

`APP_ORIGIN`

- **Função:** origem canônica (`https://dominio`, sem barra final) usada nos links de e-mail e como **RP ID** do WebAuthn.

- **Se faltar:** o sistema usa a origem da requisição. Funciona, mas você perde a defesa contra *host-header injection* e, com passkey ligada, prende as credenciais ao host usado no cadastro.
- **Pode trocar depois? Não**, se houver passkeys registradas — trocar invalida todas.
- **Obrigatória** quando `exigir_passkey_assinatura` estiver ligada.

13. Catálogo B — os segredos que protegem em silêncio

Este é o grupo mais importante da apostila. **Nenhuma destas variáveis quebra o sistema quando falta.** Todas degradam uma proteção sem emitir erro.

`PASSWORD_PEPPER` 🔒 *load-bearing*

- **Função:** segredo global aplicado por HMAC-SHA256 sobre a senha **antes** do PBKDF2 (formato `pbkdf2v3`).
- **Por que existe:** o runtime da Cloudflare impõe teto rígido de **100 000 iterações** no PBKDF2 — pedir mais lança erro. O pepper compensa isso com custo de CPU praticamente zero.
- **Se faltar:** as senhas ficam em PBKDF2 de 100k sem pepper (`pbkdf2v2`). Um vazamento do banco permite **força bruta offline de toda a base** com hardware comum.
- **Comportamento na migração:** com o pepper definido, hashes `v1` / `v2` sobem para `v3` progressivamente, no login de cada usuário. Sem *big bang*.
- **Pode trocar depois? NUNCA.** Trocar invalida todos os hashes `v3` — toda a corporação precisaria redefinir a senha.
- ☒ O login de break-glass (`SUPER_ADMIN` / `ADMIN_GERAL` por env) **não** depende do pepper, então o acesso root sobrevive mesmo à perda do segredo.

`CPF_ENCRYPTION_KEY` 🔒 *load-bearing*

- **Função:** cifra o CPF em repouso com AES-256-GCM (prefixo `enc:v1:`) na coluna `policiais.cpf` .
- **Se faltar:** o CPF é gravado **em texto puro**, em silêncio. É *fail-open* deliberado (mantém o app de pé), e foi exatamente por isso que o `/api/health?detail=` passou a reportar a ausência: dava para popular um banco inteiro assim e só descobrir num incidente.
- **Pode trocar depois? Não.** Trocar torna os CPFs já cifrados indecifráveis; ligar **depois** de popular exige re-cifrar ou zerar e resincronizar.

`CPF_INDEX_KEY` 🔒 *load-bearing*

- **Função:** HMAC-SHA256 que gera o **índice cego** (`cpf_index`), permitindo `WHERE cpf_index = ?` sem decifrar a tabela.
- **Deve ser DIFERENTE** da `CPF_ENCRYPTION_KEY` .
- **Se faltar:** o **login por certificado digital não acha o titular** — o e-CPF autentica e o sistema não sabe de quem é.

`AUDIT_CHAIN_KEY` 🔒 *load-bearing*

- **Função:** faz a cadeia de hash da trilha de auditoria usar HMAC-SHA256 em vez de SHA-256 puro.

- **Se faltar:** a cadeia detecta adulteração **acidental**, mas quem tem acesso de escrita ao banco **forja a cauda inteira** — e o console continua dizendo "cadeia íntegra".
- **Como conferir que pegou:** `/auditoria` → **Verificar integridade**. O resultado informa o modo: `HMAC-SHA256` (correto), `SHA-256 puro` (a chave não está definida) ou `misto` (adotada no meio da vida do log — as linhas anteriores seguem forjáveis).
- **Pode trocar depois?** Não: as linhas antigas ficam inverificáveis.

AUDIT_IP_ENCRYPTION_KEY 🔒

- **Função:** grava o IP **completo** de cada evento cifrado em `audit_log.ip_cifrado`, decifrável só em perícia autorizada. A coluna `ip` continua anonimizada (/24, /64) para exibição.
- **Se faltar:** só o IP anonimizado é preservado — a perícia autorizada perde o dado.
- **Deve ser distinta** das chaves de CPF.

RATE_LIMIT_IP_SALT 🔒

- **Função:** hasheia o IP completo com salt antes de gravá-lo nas tabelas de rate-limit.
- **Se faltar:** o rate-limit cai para um prefixo de IP (/24). Numa delegacia, onde todo mundo sai pelo mesmo NAT, **cinco falhas de login bloqueiam a unidade inteira** — DoS barato e lockout mútuo.
- **Pode trocar depois?** Sim. Trocar apenas reseta as janelas de rate-limit em andamento.

💡 **Como conferir as seis de uma vez:** GET `/api/health?detail=<HEALTH_DETAIL_TOKEN>` traz, dentro de `checks`, uma entrada por segredo (`senhaPepper`, `cpfCifrado`, `cpfIndice`, `auditCadeia`, `auditIp`, `rateLimitIpSalt`) com `ok` ou `ausente` — e a lista `protecoesAusentes` com os que faltam. Nunca o valor. É a forma de auditar a instalação sem abrir o painel. Ver capítulo 21.

14. Catálogo C — e-mail

O sistema tem dois provedores e escolhe o padrão em **Configurações Gerais** (`email.provedor_padrao`, default `cloudflare`); **o outro assume automaticamente quando o padrão falha**, inclusive por estouro de cota.

Binding EMAIL (Cloudflare Email Sending)

- **O que é:** não é variável de ambiente — é um **binding**, configurado no painel do projeto (Settings → Functions/Bindings → Email) ou no `wrangler.toml`.
- **Se faltar:** o caminho Cloudflare tenta a API REST (ver abaixo) e, falhando, cai no Resend.
- ⚠️ **Armadilha do remetente:** o endereço de envio é **fixo no código** — `sistema@nao-responda.escalaspcce.com.br`, constante `CF_FROM` em `src/lib/server/email.ts`. Se esse domínio não estiver na sua conta Cloudflare com Email Sending habilitado, o envio falha sempre **e o sistema cai no Resend** — que funciona, e por isso ninguém investiga o log `[email/cloudflare]`. Para usar domínio próprio, altere a constante no mesmo PR em que configurar o Email Sending. Não há variável de ambiente para isso de propósito: o serviço só entrega de domínio verificado na conta, então um valor configurável só moveria a falha para o runtime (o motivo está escrito na própria constante).

CLOUDFLARE_API_TOKEN + CLOUDFLARE_ACCOUNT_ID

- **Função:** caminho alternativo de envio pela **API REST** da Cloudflare, usado quando o binding **EMAIL** não está presente.
- **Permissão necessária no token:** envio de e-mail (Email Sending — Edit).
- **Se faltarem:** sem binding e sem esse par, o caminho Cloudflare lança **CF_EMAIL_BINDING_ABSENT** e o sistema vai para o Resend.
- ⚠ Este token é diferente do token de deploy do GitHub. Não reaproveite escopos por comodidade.

RESEND_API_KEY e RESEND_FROM_EMAIL

- **Função:** provedor alternativo (ou principal) de e-mail transacional.
- **RESEND_FROM_EMAIL :** remetente. No plano gratuito, **sem domínio verificado**, precisa ser exatamente **onboarding@resend.dev** (é o default do código). Com domínio verificado no Resend, use o endereço institucional.
- **Se faltarem:** se o caminho Cloudflare também não funcionar, **não há envio de e-mail** — e o login trava (2FA *fail-closed*).

Como testar antes do go-live

1. Crie ou escolha **uma** conta de teste com e-mail real.
2. Faça o ciclo completo: primeiro acesso → definir senha → verificar e-mail → 2FA → login → **logout** → **login de novo**.
3. O segundo login é o que confirma o caminho **v3** + pepper de ponta a ponta.

⚠ Só rode o reset em massa de senhas (capítulo 30) **depois** desse teste passar. Resetar a base inteira com o e-mail quebrado deixa todo mundo fora do sistema ao mesmo tempo.

15. Catálogo D — contas de bootstrap

Os administradores de banco (**administradores**) **nascem só destes bootstraps** — não existe tela para criá-los.

SUPER_ADMIN_LOGIN / SUPER_ADMIN_SENHA e SUPER_ADMIN_EMAIL

- **Função:** a conta root (break-glass). É quem promove administradores, gerencia policiais e unidades e configura a política de assinatura.
- **SUPER_ADMIN_SENHA aceita dois formatos:** texto claro (legado) ou **hash PBKDF2 pbkdf2v2:** — ☒ use o hash: quem lê o painel ou o **wrangler** não vê a senha. O apêndice E traz o script que gera o hash.
- **Por que o hash é v2 e não v3 :** de propósito. O break-glass **não** depende do **PASSWORD_PEPPER**, então a conta root entra mesmo se o pepper for perdido.
- **SUPER_ADMIN_EMAIL :** opcional no código, **obrigatório na prática**. Com ele, o login root passa a exigir 2FA.
- **Se faltar o e-mail:** o login da conta mais poderosa do sistema entra **direto, só com a senha da env**. É o break-glass funcionando como projetado — e um risco enorme se ficar assim em produção.
- **Se faltar a conta inteira:** você não tem como criar administradores. Não há caminho pela interface.
- 💡 Todo login por bootstrap é auditado e gera um evento **warning** no Sentry.

ADMIN_GERAL_LOGIN / ADMIN_GERAL_SENHA / ADMIN_GERAL_EMAIL

- **Função:** conta de Admin Geral por variável de ambiente, para o setup inicial.
- **Mesma regra de hash e de 2FA** do Super Admin.
-  **O bootstrap se autodesliga:** assim que o registro do admin no banco tiver um `email` configurado, o login por env é **bloqueado**, com um erro explícito no log dizendo para remover as variáveis. Isso é intencional — a partir daí, entre pelo fluxo normal com 2FA.
-  Depois do setup, **remova** `ADMIN_GERAL_LOGIN` e `ADMIN_GERAL_SENHA` do ambiente.

16. Catálogo E — webhooks e operações destrutivas

**RESET_TOKEN **

- **Função:** segredo **separado** exigido pelo endpoint `/api/webhook/reset-policiais`, que **apaga todas as tabelas operacionais**.
- **Se faltar:** o endpoint devolve 401 para qualquer requisição. Ou seja, **não configurar é a opção segura** — é *fail-closed*.
-  **Precisa ser estritamente diferente do SYNC_TOKEN.** O desenho existe para que comprometer o token de webhook não baste para apagar o banco. Igualá-los anula a separação inteira.
- **As quatro camadas do endpoint:** `Authorization: Bearer <SYNC_TOKEN> + X-Reset-Token + X-Confirm-Reset: <AAAA-MM-DD em UTC> + X-Webhook-Timestamp / X-Webhook-Nonce` (anti-replay, sempre obrigatórios aqui).

WEBHOOK_REPLAY_ENFORCE

- **Função:** quando `truthy (1, true, yes, on)`, os webhooks **recusam** requisições sem os headers de anti-replay.
- **Se faltar:** o sistema aceita chamadas sem os headers (compatibilidade), apenas registrando um `info` no log. Uma requisição capturada pode ser reproduzida.
- **Ordem de ativação (importa):** (1) faça o deploy do código; (2) **republique a Web App do Apps Script**, que é o que faz o emissor passar a mandar os headers; (3) só então ligue a flag. Inverter a ordem derruba a sincronização com 401.
-  Em produção: `WEBHOOK_REPLAY_ENFORCE=1`.
-  O endpoint destrutivo exige os headers **independentemente** desta flag.

WEBHOOK_ALLOW_PAPEL_CHANGES

- **Função:** permite que o webhook de sincronização altere `papel` e `papel_unidade_id` dos policiais.
- **Se faltar (default vazio):** o webhook **não** altera papéis — preserva o que está no banco. É o que impede que um `SYNC_TOKEN` comprometido promova qualquer matrícula a `admin_seccional` pela planilha.
-  Em produção: **deixe vazio**, a menos que a planilha seja a fonte canônica também para papel. Promoções legítimas acontecem em `/policiais/[id]`.

17. Catálogo F — assinatura digital

Este grupo decide o **valor jurídico** dos documentos que o sistema emite. Configure com calma e na ordem apresentada.

17.1 Trust store ICP-Brasil

Antes das variáveis, um passo de arquivo. A verificação da cadeia depende de dois PEMs versionados:

```
src/lib/server/assinatura/icp-brasil/roots.pem
src/lib/server/assinatura/icp-brasil/intermediates.pem
```

💡 **No repositório atual eles já estão populados** (atualizados em 03/09/2026). Confira antes de assumir que precisa gerá-los:

```
wc -c src/lib/server/assinatura/icp-brasil/*.pem
head -3 src/lib/server/assinatura/icp-brasil/roots.pem
```

Se estiverem vazios (instalação nova a partir de um repositório zerado), popule:

```
cd src/lib/server/assinatura/icp-brasil
./update-trust-store.sh          # baixa raízes da ITI + ZIP das ACs credenciadas
git diff roots.pem intermediates.pem # confira o que mudou
git add roots.pem intermediates.pem
git commit -m "chore(icp-brasil): popula trust store ($(date +%F))"
```

💡 O `.env.example` citava o caminho antigo `src/lib/server/icp-brasil`; foi corrigido para `src/lib/server/assinatura/icp-brasil`, que é onde os arquivos vivem.

💡 Existe um workflow mensal (`update-icp-brasil-trust-store.yml`) que roda esse script sozinho e **abre um PR** quando a ITI publica mudanças. Capítulo 27.

ICP_BRASIL_TRUST_STORE_REQUIRED

- **Função:** quando `truthy`, o sistema **recusa** finalizar assinatura qualificada se o trust store estiver vazio.
- **Se faltar:** o sistema apenas registra um *warning* e **aceita** a assinatura. Em produção isso é grave: sem cadeia validada, um certificado autoassinado passaria como "qualificada ICP-Brasil".
- ☒ Em produção: `ICP_BRASIL_TRUST_STORE_REQUIRED=1`, depois de popular os PEMs.

17.2 Carimbo de tempo (TSA)

TSA_URL

- **Função:** endpoint RFC 3161 que carimba a assinatura, promovendo CAdES-BES → CAdES-T.

- **Default embutido:** o `wrangler.toml` define `http://timestamp.digicert.com` — público, gratuito, **não é ACT ICP-Brasil**.
- **Se faltar:** o sistema usa só o `signingTime` do próprio servidor — **sem oponibilidade a terceiros** conforme o DOC-ICP-15.
- **Para peso ICP pleno:** aponte para uma ACT credenciada (SERPRO, Bry, Soluti, Certisign, AC Safeweb, ICP-EDU). São serviços **pagos**.
- 💡 Sobre o `http://` do default: foi medido, não é descuido. O endpoint da DigiCert responde em HTTP e **reseta a conexão em HTTPS**. O risco é limitado — o carimbo é assinado pela TSA, então um intermediário não forja carimbo válido; consegue apenas negá-lo (a assinatura cai para o horário do servidor). ACTs ICP-Brasil publicam endpoint HTTPS; use-o.

TSA_USERNAME / TSA_PASSWORD 🔒

- **Função:** autenticação HTTP Basic, se a ACT exigir.
- **Se faltarem** (com ACT que exige): o carimbo falha e a assinatura é rebaixada — ou recusada, se a flag abaixo estiver ligada.

EXIGIR_TSA_QUALIFICADA

- **Função:** recusa finalizar a assinatura se o carimbo não for de ACT credenciada verificada (`act_icp`).
- **Se faltar:** carimbo não-ICP ou ausente apenas **rebaixa o rótulo** (`tlsa_externa` / `servidor`), sem bloquear.
- ⚠️ **A armadilha mais cara desta apostila:** ligar `EXIGIR_TSA_QUALIFICADA=1` **sem** trocar a `TSA_URL` faz o sistema **rejeitar 100% das assinaturas qualificadas com HTTP 422** — porque o carimbo da DigiCert nunca vira `act_icp`. O finalizador detecta a combinação e emite `[CADES][CONFIG]` no log; configure um alerta no Sentry para essa string.
- ✅ Ordem correta: contratar a ACT → configurar `TSA_URL` (+ credenciais) → testar uma assinatura em staging → só então ligar a flag.

17.3 Selo institucional (assinatura avançada)

SELO_INSTITUCIONAL_PEM 🔒 *load-bearing*

- **Função:** bundle base64 (chave privada PEM + certificado PEM) do selo autoassinado que sela os PDFs assinados **em tela** (assinatura avançada, Lei 14.063/2020 art. 4º II).
- **Como gerar:**

```
node scripts/gerar-selo-institucional.mjs "Sistema de Escalas - PCCE" "Polícia Civil do Ceará"
```

O script produz três coisas: `selo-institucional.key.pem` (🔒 privada, *gitignored*, guarde em cofre), `selo-institucional.cert.pem` (público, versionado para conferência) e, no stdout, o **bundle base64** que vai na variável.

```
wrangler pages secret put SELO_INSTITUCIONAL_PEM --project-name=escalas
# cole o bundle base64 quando pedir
```

- **Se faltar:** o PDF da assinatura em tela degrada para um rodapé honesto, **sem selo** — não vira um CMS autocontido à prova de adulteração.
- **Pode trocar depois?** Tecnicamente sim, mas trocar **muda a identidade do selo**. Os PDFs antigos continuam verificáveis pelo certificado público antigo — que você precisa preservar. O script **aborta** se os arquivos já existirem, justamente para você não trocar a identidade por acidente (`--force` sobrescreve).
- 💡 Não é ICP-Brasil: o Adobe mostrará "validade desconhecida". A confiança vem do portal `/validar` e da publicação do certificado público.

17.4 Ajustes avançados (deixe como estão)

EMBED_PADES_LT_DSS

- **Função:** anexa um dicionário DSS (certificados + OCSP) à assinatura qualificada, para LTV autocontido.
-  **Deixe vazio em produção.** As assinaturas usam o SubFilter legado `adbe.pkcs7.detached`, e o Adobe **não** reconhece o DSS anexado como LTV — passa a marcar a assinatura como "inválida", apesar da criptografia íntegra. A validade jurídica não depende disso.

PA_AD_RB_HASH_HEX

- **Função:** hash da Política de Assinatura (PA-AD-RB v2.3) usado nos SignedAttributes.
- **Se faltar:** o valor correto **já vem embutido no código**. Só defina para sobrescrever num bump de versão da política.
- ⚠️ Não é o hash do PDF do DOC-ICP-15.03 nem o hash do arquivo `.der` — é o `signPolicyHash` interno. O placeholder de zeros é explicitamente rejeitado.

18. Catálogo G — observabilidade e ajuste fino

SENTRY_DSN / SENTRY_ENVIRONMENT / SENTRY_TRACES_SAMPLE_RATE

- **Função:** captura de exceções do **servidor** (Worker).
- **Se faltar:** o SDK roda em modo *no-op* — nenhum erro 5xx é reportado. O sistema funciona; você fica cego. O `.env.example` classifica como **exigida em produção**.
- `SENTRY_ENVIRONMENT`: rótulo (`production`, `staging`). Default `production`.
- `SENTRY_TRACES_SAMPLE_RATE`: default `0.1`.

PUBLIC_SENTRY_DSN / PUBLIC_SENTRY_ENVIRONMENT

- **Função:** captura de erros de **JavaScript no navegador** — inclusive do fluxo de assinatura, que é onde mais dói.
- **É público por natureza** (vai para o cliente); pode ser o mesmo DSN.
- **Se faltar:** o Sentry do cliente fica *no-op*. Erros do painel de assinatura acontecem e ninguém fica sabendo.
- 💡 O host do Sentry já está liberado no `connect-src` da CSP.

HEALTH_DETAIL_TOKEN

- **Função:** libera o detalhe do `/api/health` via `?detail=<token>`.

- **Se faltar:** o endpoint devolve só `{ status }` — binário, anti-reconhecimento. Você perde o painel de conferência de segredos e o failsafe da limpeza de retenção (capítulo 21).
- **Mínimo:** 16 caracteres. Gere com `openssl rand -hex 32`.
- **Pode trocar depois?** Sim, sem consequência.

SESSION_CACHE_TTL_SECONDS

- **Função:** TTL do cache de sessão na borda, que evita consultar o D1 em toda requisição autenticada.
- **Default:** 60 s. **Faixa:** 0 a 300 (valores fora são truncados).
- **Se faltar:** vale o default de 60 s — é uma escolha razoável.
- **Trade-off:** o valor é a janela em que um logout, uma desativação de usuário ou uma troca de papel ainda podem valer em outro data center. `0` desliga o cache (revogação imediata, mais consultas ao D1).
- 💡 Quem **muda** nunca usa o cache: o TTL é zerado para `POST / PUT / PATCH / DELETE`. O atraso vale só para leitura.

19. Catálogo H — integração com a planilha Google

Opcional, mas é assim que o cadastro de servidores costuma ser alimentado.

GISE_BASE_EQUIPE_WEBHOOK_URL

- **Função:** URL do **Web App** do Apps Script que grava a aba `Base_Equipe` quando uma escala extra é finalizada.
- **Formato:** `https://script.google.com/macros/s/AKfy.../exec`.
- ⚠️ **Não** use a URL da planilha (`docs.google.com/spreadsheets/...`) nem a do portal (`*.pages.dev`): o POST iria para o site e voltaria como redirecionamento para `/login`.
- **Se faltar:** o envio automático e o botão "Enviar para a planilha" respondem erro de configuração. A finalização da escala **não** é desfeita.

GISE_BASE_EQUIPE_SECRET 🔒

- **Função:** segredo compartilhado com `ScriptProperties.BASE_EQUIPE_SECRET` na planilha.
- **Precisa ser o mesmo valor dos dois lados.**
- **Se faltar:** a integração não autentica.

Passo a passo da planilha

1. Na planilha: **Extensões** → **Apps Script**, cole o conteúdo de `scripts/GoogleAppsScript_Sync.gs`.
2. Recarregue a planilha — surge o menu "🚀 **Sincronização D1**".
3. Menu → "⚙️ **Configurar tokens**" → cole o `SYNC_TOKEN` (e o `RESET_TOKEN`, se for usar o reset).
4. Menu → "**Secret Base_Equipe (portal)**" → cole o mesmo valor de `GISE_BASE_EQUIPE_SECRET`.
5. Se o script não estiver vinculado ao arquivo, menu → "**ID planilha Base_Equipe (portal)**" → cole o ID da URL da planilha.
6. **Implante como Web App** ("executar como: Eu") e copie a URL `/exec` para `GISE_BASE_EQUIPE_WEBHOOK_URL`.

7. Confirme que a aba `Base_Equipe` existe, com as colunas A–J do modelo documentado no `.gs`.



Abrir a URL `/exec` no navegador pode mostrar "Script function not found: doGet" — é esperado: o portal usa POST.



Depois de alterar o `.gs`, **implante uma nova versão**. E lembre: republicar a Web App é o que faz o emissor passar a mandar os headers de anti-replay — pré-requisito para ligar `WEBHOOK_REPLAY_ENFORCE=1`.

20. Tabela-resumo de todas as variáveis

Ordem: obrigatoriedade decrescente. "Cofre" marca o que você não consegue recuperar depois.

Variável	Obrigatória?	Cofre	Se faltar
SYNC_TOKEN	Sim		Webhooks 401; sincronização e limpeza de retenção param
Binding EMAIL ou RESEND_API_KEY	Sim		Ninguém loga (2FA fail-closed) e o primeiro acesso trava
RESEND_FROM_EMAIL	Com Resend	—	Cai no default onboarding@resend.dev
APP_ORIGIN	Recomendada / obrigatória com passkey	—	Usa a origem da requisição; credencial presa ao host
PASSWORD_PEPPER	Recomendada forte		Força bruta offline viável se o banco vazar
CPF_ENCRYPTION_KEY	Recomendada forte		CPF gravado em texto puro, em silêncio
CPF_INDEX_KEY	Recomendada forte		Login por certificado não acha o titular
AUDIT_CHAIN_KEY	Recomendada forte		Trilha de auditoria forjável por quem escreve no banco
AUDIT_IP_ENCRYPTION_KEY	Recomendada		Perícia perde o IP completo
RATE_LIMIT_IP_SALT	Recomendada		5 falhas bloqueiam a delegacia inteira (NAT)
SUPER_ADMIN_LOGIN / SENHA	Sim, no setup		Não há como criar administradores
SUPER_ADMIN_EMAIL	Recomendada forte	—	Login root sem 2FA
ADMIN_GERAL_LOGIN / SENHA	Opcional		— (remover após o setup)
ADMIN_GERAL_EMAIL	Recomendada	—	Login de bootstrap sem 2FA
RESET_TOKEN	Só se usar o reset		Endpoint destrutivo responde 401 (seguro)
WEBHOOK_REPLAY_ENFORCE	=1 em produção	—	Requisição capturada pode ser reproduzida
WEBHOOK_ALLOW_PAPEL_CHANGES	Vazia em produção	—	(ligada) planilha comprometida promove admins
HEALTH_DETAIL_TOKEN	Recomendada		Sem painel de conferência e sem failsafe do cron
SESSION_CACHE_TTL_SECONDS	Opcional	—	Default 60 s
ICP_BRASIL_TRUST_STORE_REQUIRED	=1 em produção	—	Cert autoassinado passa como "ICP-Brasil"
TSA_URL	Recomendada	—	Só signingTime do servidor, sem oponentabilidade
TSA_USERNAME / TSA_PASSWORD	Se a ACT exigir		Carimbo falha

Variável	Obrigatória?	Cofre	Se faltar
EXIGIR_TSA_QUALIFICADA	Recomendada com ACT	—	Carimbo não-ICP só rebaixa o rótulo
SELO_INSTITUCIONAL_PEM	Sim, se usar avançada		PDF sem selo (rodapé honesto)
EMBED_PADES_LT_DSS	Vazia	—	(ligada) Adobe marca a assinatura como inválida
PA_AD_RB_HASH_HEX	Não	—	Valor oficial já embutido no código
SENTRY_DSN	Recomendada forte	—	5xx do servidor perdidos
SENTRY_ENVIRONMENT / SENTRY_TRACES_SAMPLE_RATE	Opcionais	—	Defaults production / 0.1
PUBLIC_SENTRY_DSN / PUBLIC_SENTRY_ENVIRONMENT	Recomendadas	—	Erros de JS do cliente perdidos
CLOUDFLARE_API_TOKEN / CLOUDFLARE_ACCOUNT_ID	Se usar e-mail por API	 (token)	Caminho REST de e-mail indisponível
GISE_BASE_EQUIPE_WEBHOOK_URL	Se usar a planilha	—	Envio da Base_Equipe falha
GISE_BASE_EQUIPE_SECRET	Se usar a planilha		Integração não autentica

21. Conferindo o que realmente pegou

Depois de configurar tudo e fazer um deploy, **não confie no painel** — pergunte ao sistema.

```
curl -s "https://<seu-dominio>/api/health" | jq
# → { "status": "ok" }   (público, binário, de propósito)

curl -s "https://<seu-dominio>/api/health?detail=<HEALTH_DETAIL_TOKEN>" | jq
```

A resposta detalhada tem esta forma:

```
{
  "status": "healthy",
  "checks": {
    "database": "ok",
    "r2": "ok",
    "limpezaRetencao": "ok",
    "cpfCifrado": "ok",
    "cpfIndice": "ok",
    "senhaPepper": "ok",
    "auditCadeia": "ok",
    "auditIp": "ok",
    "rateLimitIpSalt": "ok"
  },
  "retencao": { "ultimaExecucao": "...", "horasDesdeUltima": 3.1, "atrasada": false },
```



```
"protecoesAusentes": []
}
```

Campo	O que diz
status	healthy ou degraded — atenção: só a resposta pública usa ok / down
checks.database / checks.r2	Os bindings respondem?
checks.limpezaRetencao	ok ou stale (cron atrasado)
Os seis segredos dentro de checks	ok ou ausente — nunca o valor
retencao	Quando a limpeza rodou e se está atrasada
protecoesAusentes	A lista dos segredos que faltam. Vazia é o alvo

💡 Note que status fica degraded quando há proteção ausente **ou** o cron está atrasado, mesmo com D1 e R2 saudáveis. Isso é intencional: proteção desligada é degradação.

✅ **O critério de aprovação da implantação é este endpoint com protecoesAusentes vazio.** É a única forma de saber, de fora, que a cifragem em repouso e a cadeia de auditoria estão realmente ligadas.

E uma conferência que só existe dentro do sistema: entre como Super Admin em /auditoria → **Verificar integridade**. O resultado tem de dizer **HMAC-SHA256**. Se disser **SHA-256 puro**, o **AUDIT_CHAIN_KEY** não pegou; se disser **misto**, a chave entrou depois de o log já existir — as linhas anteriores seguem forjáveis.

Parte III — GitHub Actions

22. Os quatro workflows

Workflow	Quando roda	O que faz	Se não configurar
<code>deploy.yml</code>	Push/PR em <code>main</code> e <code>staging</code>	Testes, guards, migrações e deploy	Não há deploy
<code>cleanup-retencao.yml</code>	Diário, 04:17 UTC	Chama a limpeza de retenção (LGPD)	Tabelas crescem em silêncio e consomem cota do D1
<code>backup-d1.yml</code>	Diário, 05:43 UTC	Exporta, cifra e guarda o dump do banco no R2	Sem backup lógico — só o Time Travel de ~30 dias
<code>update-icp-brasil-trust-store.yml</code>	Mensal, dia 1	Atualiza a cadeia da ITI e abre PR	O trust store envelhece; certificado novo da ITI não valida

💡 Os horários são "quebrados" de propósito, para fugir do pico de agendamentos do GitHub em `:00`.

⚠️ **O Cloudflare Pages não tem cron.** Se você desabilitar o Actions (repositório arquivado, limite de minutos, workflow desligado à mão), a limpeza e o backup param — e nada na aplicação avisa. O capítulo 26 mostra como monitorar isso.

23. Onde ficam os segredos do GitHub

Caminho: **repositório** → **Settings** → **Secrets and variables** → **Actions**.

Ali existem três coisas diferentes, e confundi-las é a origem de metade dos problemas:

Tipo	Aba	Visível?	Use para
Repository secret	Secrets → Repository secrets	Não (mascarado no log)	Segredos usados por qualquer workflow
Environment secret	Secrets → Environment secrets	Não	Segredos de um ambiente só (<code>production</code> , <code>staging</code>)
Variable	Variables	Sim , aparece no log	Valores não sensíveis

Regras práticas deste projeto:

- Tudo que esta apostila lista como secret vai em **Repository secrets** (é o que os workflows esperam: `${{ secrets.X }}` resolve tanto repository quanto environment secret).
- Se você quiser tokens diferentes para produção e staging, mova `CLOUDFLARE_API_TOKEN` para **Environment secrets** dos ambientes correspondentes. O `deploy.yml` já declara `environment: production` e `environment: staging` nos jobs de deploy, então a resolução funciona sem alterar o workflow.
- ⚠️ **O Actions mascara segredos no log**, mas o mascaramento é por correspondência de texto. Um segredo interpolado dentro de um script grande pode escapar do mascaramento se for quebrado ou transformado. Por isso

os workflows deste projeto passam segredos por `env:` em vez de interpolar no corpo do script — mantenha esse padrão ao editar.

24. Environments: o portão de produção

Caminho: **Settings** → **Environments**.

Crie dois: `production` e `staging`. Eles já são referenciados pelos jobs de deploy — se não existirem, o GitHub os cria na primeira execução, mas criá-los antes permite configurar proteção.

O que vale a pena ligar em `production`:

Proteção	Efeito
Required reviewers	O job de deploy espera aprovação humana antes de publicar. Vira o "botão de deploy"
Wait timer	Atraso antes de publicar — janela para cancelar
Deployment branches	Só <code>main</code> pode publicar em <code>production</code>

💡 Como o deploy roda migração de banco **antes** do `pages deploy`, um *required reviewer* em `production` é também um portão sobre a migração — que é a operação verdadeiramente irreversível.

25. `deploy.yml` — o pipeline principal

25.1 Secrets necessários

Secret	Valor	Se faltar
<code>CLOUDFLARE_API_TOKEN</code>	O token de deploy do capítulo 9.1	O deploy falha com 403 (ou a migração falha antes, se faltar a permissão de D1)
<code>CLOUDFLARE_ACCOUNT_ID</code>	O ID do capítulo 5	Wrangler não resolve a conta

São só esses dois. Todo o resto do pipeline (testes, E2E, guards) roda **sem segredo nenhum** — a suíte E2E gera o próprio `.dev.vars` com um token de teste.

25.2 O que o pipeline faz, na ordem

O job `test` roda em todo push e em todo PR para `main` / `staging`:

#	Passo	Falha significa
1	<code>npm ci + svelte-kit sync</code>	Dependência quebrada
2	<code>npm run lint:ci (--max-warnings 0)</code>	Warning novo de ESLint
3	<code>format:check</code> e <code>format:check:e2e</code>	Rode <code>npm run format / format:e2e</code>
4	<code>npm run knip</code>	Código ou export morto
5	<code>svelte-check --threshold error</code>	Erro de tipo
6	<code>vitest run --coverage</code>	Teste unitário quebrado
7	<code>npm run build</code>	Build quebrado
8	Guard — convenção de testes	<code>*.test.ts</code> fora de <code>__tests__</code> /
9	Guard — padrão de erros de API	<code>return json({ error</code> numa rota
10	Guard — permissão de documento assinado	Rota crítica sem o helper de permissão
11–14	<code>guard:autorizacao</code> , <code>guard:duplicacao</code> , <code>guard:achados</code> , <code>guard:entrada</code>	Violação de padrão do projeto
15	<code>docs:guard</code>	Arquivo novo em <code>lib/db</code> sem documentação
16	Migrações D1 locais + Playwright	E2E quebrado

Os jobs `deploy-staging` e `deploy-production` rodam **só em push** (nunca em PR), cada um na sua branch, e fazem: `npm ci` → `npm run build` → **migração do D1 do ambiente** → `wrangler pages deploy`.

25.3 Proteção de branch

Caminho: **Settings** → **Branches** → **Add branch ruleset** (ou *Add rule*, em interfaces mais antigas).

Configure para `main`:

- ✓ **Require a pull request before merging**
- ✓ **Require status checks to pass** → selecione o check `test`
- ✓ **Require branches to be up to date before merging**
- ✓ **Do not allow bypassing the above settings** (inclusive para administradores)

Se faltar: um push direto em `main` publica em produção sem passar por revisão. O pipeline ainda roda os testes — mas o deploy não espera por eles em jobs paralelos de outras execuções, e nada impede o merge de um PR vermelho.

25.4 Duas decisões do workflow que você não deve "otimizar"

⚠ **cancel-in-progress** só vale em **pull request**, nunca em **push**. Em `main`, dois merges seguidos ficariam no mesmo grupo de concorrência e o segundo cancelaria o primeiro — que é justamente o que roda a migração de produção. O `migrate.ts` grava em `_migrations_aplicadas` **depois** do `d1 execute`: morrer nessa janela deixa a migração aplicada e **não registrada**, o que a faz rodar de novo no próximo deploy.

⚠ **Não passe o diretório de build como artefato entre jobs** para "economizar" o segundo build. Já foi tentado e quebrou o deploy de produção: a saída do adapter não é autocontida — ela importa arquivos de fora do diretório publicado, incluindo um temporário do próprio adapter.

26. `cleanup-retencao.yml` — a limpeza da LGPD

26.1 Secrets

Secret	Valor	Detalhe crítico
APP_BASE_URL	URL pública de produção , sem barra final	Ex.: <code>https://escalas.suainstituicao.gov.br</code>
SYNC_TOKEN	O mesmo valor já configurado no Cloudflare Pages (Production)	Não gere um novo aqui

⚠ `SYNC_TOKEN` aqui **não é um token novo** — o aplicativo compara com o dele. Gerar outro é a forma mais comum de tomar 401.

⚠ Uma barra final no `APP_BASE_URL` viraria `//api/webhook/...`. O workflow remove a barra por precaução, mas não conte com isso em outros lugares.

26.2 Por que ele pode dar 401 mesmo com os secrets certos

O workflow envia `X-Webhook-Timestamp` e `X-Webhook-Nonce` em toda chamada porque `WEBHOOK_REPLAY_ENFORCE=1` está **ligado em produção** — sem os dois headers, o endpoint recusa mesmo com o token correto. Configurar os secrets **não basta** por si só; é o par de headers que completa a autenticação.

Detalhe elegante do desenho: o **nonce é novo a cada tentativa**, e é por isso que a retentativa é um laço explícito em vez de `curl --retry`. Reenviar o mesmo nonce voltaria como 401 de replay, o que apontaria "token errado" quando o problema foi uma indisponibilidade passageira.

O próprio workflow, ao receber 401, imprime as três causas prováveis:

1. `SYNC_TOKEN` do GitHub diferente do configurado no Cloudflare (Production);
2. `APP_BASE_URL` apontando para um deployment de **preview**, que tem outro secret;
3. o token tem menos de 32 caracteres (recusa *fail-closed*).

26.3 O failsafe: monitore o atraso

Se o cron parar, nada quebra imediatamente — as tabelas apenas crescem. Para flagrar:

```
curl -s "https://<seu-dominio>/api/health?detail=<HEALTH_DETAIL_TOKEN>" | jq .retencao
# { "ultimaExecucao": "...", "horasDesdeUltima": 73.2, "atrasada": true }
```

✓ **Aponte um monitor externo** (UptimeRobot, Better Stack, o que a instituição já usar) para essa URL e alerte quando `retencao.atrasada` for `true` ou `status` for `degraded`. A tolerância padrão é 48 h (o cron roda a cada 24 h). A liveness pública **não** muda por causa do atraso — continua `200 ok`.

26.4 O número que você monitora depois

A resposta do webhook traz `pendenciasRestantes` — eventos de auditoria que a cadeia recusou. **Zero é o normal**. Crescendo entre execuções, há evento que nunca vai entrar sozinho:

```
SELECT acao, entidade, motivo, tentativas, created_at
FROM audit_pendencias ORDER BY tentativas DESC, created_at;
```

Um `tentativas` alto com o mesmo `motivo` é defeito, não ruído — a trilha está incompleta até alguém agir.

27. `backup-d1.yml` — o backup cifrado

27.1 O que ele faz

`wrangler d1 export` → conferência de sanidade (piso de tamanho e presença de tabelas-chave: um dump vazio **falha o job** em vez de subir silencioso) → `gzip` → **cifra com age** → grava no bucket privado `escalas-backups`:

- `d1/diario/backup-d1-AAAA-MM-DD.sql.gz.age` — um por dia;
- `d1/mensal/backup-d1-AAAA-MM.sql.gz.age` — snapshot do dia 1º.

O dump contém dado pessoal em claro (nome, e-mail, telefone de milhares de servidores), por isso é cifrado **antes de sair do runner** e nunca é anexado como artefato do Actions.

27.2 Setup, na ordem

1. Gerar o par de chaves `age`:

```
age-keygen -o backup-key.txt
# Public key: age1qz...
```

🔑 O arquivo `backup-key.txt` contém a **chave privada** — guarde em cofre **offline**. A linha `# public key: age1...` é o valor do secret.

⚠ **Sem a chave privada, os backups são ilegíveis.** Trate-a com o mesmo cuidado do `PASSWORD_PEPPER`.

2. Criar o bucket e configurar retenção (capítulo 7).

3. Criar o token dedicado (capítulo 9.2).

4. Cadastrar os secrets:

Secret	Valor
CLOUDFLARE_ACCOUNT_ID	O mesmo do deploy (já deve existir)
CLOUDFLARE_BACKUP_API_TOKEN	O token dedicado
BACKUP_AGE_PUBLIC_KEY	A chave pública age1...

O workflow valida os três no primeiro passo e falha com mensagem explícita se algum faltar.

5. Disparar manualmente: aba **Actions** → **Backup D1** → **Run workflow**. Confirme que o objeto apareceu no bucket.

27.3 Teste de restauração (não pule)

```
# 1. Baixar
npx wrangler r2 object get escalas-backups/d1/diario/backup-d1-AAAA-MM-DD.sql.gz.age \
  --file=backup.sql.gz.age --remote

# 2. Decifrar e descomprimir (exige a chave privada, fora do CI)
age -d -i backup-key.txt backup.sql.gz.age | gunzip > backup.sql

# 3. Aplicar num banco DESCARTÁVEL — nunca por cima de produção
npx wrangler d1 execute <db-descartavel> --remote --file=backup.sql
```

Backup que nunca foi restaurado não é backup. Faça o teste depois do setup e repita periodicamente.

27.4 O outro mecanismo: Time Travel

O D1 tem recuperação para qualquer ponto dos últimos ~30 dias, sem backup manual:

```
npx wrangler d1 time-travel info escalas-db --remote
npx wrangler d1 time-travel restore escalas-db --remote --timestamp="2026-06-05T12:00:00Z"
```

⚠ O Time Travel **substitui** o estado atual. Faça um **export** **antes** de restaurar, ou você perde tudo que foi gravado depois do ponto escolhido.

28. `update-icp-brasil-trust-store.yml` — a cadeia da ITI

28.1 O que ele faz

Roda todo dia 1º às 06:00 UTC: executa o `update-trust-store.sh`, compara os PEMs ignorando a linha de carimbo de data (senão abriria PR todo mês sem mudança real) e, havendo diferença, **abre um PR** com o diff — que um humano revisa e mergeia.

28.2 Permissões

O workflow declara `contents: write` e `pull-requests: write`. Para que ele consiga abrir PR, confirme em **Settings** → **Actions** → **General** → **Workflow permissions**:

- **Read and write permissions** habilitado, **ou** as permissões declaradas no workflow respeitadas pela política da organização;
- ☒ **Allow GitHub Actions to create and approve pull requests** marcado.

Se faltar: o job roda, detecta a mudança e falha ao abrir o PR — e a cadeia nunca é atualizada.

28.3 Notificação por e-mail (opcional)

Secret	Valor
GMAIL_USER	Conta Gmail usada como remetente SMTP
GMAIL_APP_PASSWORD	App Password do Gmail (não a senha da conta)
ICP_TRUST_NOTIFY_TO	Destinatário(s); vários separados por vírgula

O passo tem `continue-on-error: true`, então **a ausência dos secrets não quebra o workflow** — o PR é aberto de qualquer forma; só a notificação falha.

💡 A notificação vem **ligada** e se desliga pela variável de repositório `NOTIFY=false` (**Settings** → **Secrets and variables** → **Actions** → **Variables**). O gate precisa ser `vars` e não `secrets`: o contexto `secrets` não está disponível em condição de step, então não há como pular o passo "quando os secrets faltarem" — sem eles, o passo falha e o `continue-on-error` segura o workflow. O X vermelho nesse passo é o sinal de notificação configurada pela metade.

Por que a notificação importa: o PR pode ficar parado na aba Pull Requests por dias. A ITI eventualmente publica uma cadeia nova, com certificado que **só vai validar depois do merge** — e o sintoma, no sistema, é assinatura qualificada legítima sendo recusada.

29. Dependabot

Já configurado em `.github/dependabot.yml`. Ele abre PRs:

- **semanalmente** (segundas, 06:00 BRT) para dependências npm;
- **mensalmente** para actions do GitHub;
- **imediatamente** para qualquer vulnerabilidade publicada.

Para habilitar os alertas: **Settings** → **Code security and analysis** → ligue **Dependabot alerts** e **Dependabot security updates**.

⚠️ ****Três dependências têm upgrade *major* ignorado de propósito**** — `node-forge`, `pdf-lib` e `@signpdf/*`. Todas estão no caminho da assinatura digital: uma mudança de comportamento pode alterar a validação da cadeia ICP-Brasil ou quebrar PDFs já assinados. Upgrade major dessas três é feito à mão, depois de testar o fluxo de assinatura ponta a ponta em staging.

Prazo de resposta sugerido pelo runbook: vulnerabilidade `critical` em 24 h, demais em 7 dias.

30. Tabela-resumo dos secrets do GitHub

Secret	Workflow	Obrigatório?	Se faltar
<code>CLOUDFLARE_API_TOKEN</code>	<code>deploy.yml</code>	Sim	Sem deploy e sem migração
<code>CLOUDFLARE_ACCOUNT_ID</code>	<code>deploy.yml</code> , <code>backup-d1.yml</code>	Sim	Wrangler não resolve a conta
<code>APP_BASE_URL</code>	<code>cleanup-retencao.yml</code>	Sim (para a LGPD)	O workflow falha com mensagem explícita
<code>SYNC_TOKEN</code>	<code>cleanup-retencao.yml</code>	Sim (para a LGPD)	401 — e as tabelas de retenção crescem
<code>CLOUDFLARE_BACKUP_API_TOKEN</code>	<code>backup-d1.yml</code>	Sim (para backup)	Sem backup lógico
<code>BACKUP_AGE_PUBLIC_KEY</code>	<code>backup-d1.yml</code>	Sim (para backup)	Sem backup lógico
<code>GMAIL_USER</code>	<code>update-icp-...yml</code>	Não	Sem notificação (o PR ainda abre)
<code>GMAIL_APP_PASSWORD</code>	<code>update-icp-...yml</code>	Não	Idem
<code>ICP_TRUST_NOTIFY_TO</code>	<code>update-icp-...yml</code>	Não	Idem



Nenhum outro secret é necessário. Em particular, **o job de testes não precisa de segredo nenhum** — a suíte E2E monta o próprio ambiente.

Parte IV — Primeiro deploy e go-live

31. Preparar o repositório

31.1 Obter o código

```
git clone <url-do-repositorio> escalas
cd escalas
npm ci
```

31.2 Conferir o que precisa ser seu

Item	Onde	O que fazer
database_id de produção	wrangler.toml	Trocar pelo ID do seu escalas-db
database_id de staging	wrangler.toml ([[env.preview.d1_databases]])	Trocar pelo do seu escalas-db-staging
Nome do projeto Pages	deploy.yml (--project-name=escalas)	Manter escalas (ou trocar nos dois jobs)
Trust store ICP-Brasil	src/lib/server/assinatura/icp-brasil/*.pem	Conferir se está populado (capítulo 17.1)
Remetente do e-mail Cloudflare	src/lib/server/email.ts (CF_FROM)	Ajustar se o domínio for outro

Commite e faça push dessas alterações **antes** do primeiro deploy.

31.3 Validar localmente (recomendado)

```
cp .env.example .dev.vars      # preencha SYNC_TOKEN e RESET_TOKEN, no mínimo
npm run db:migrate             # cria o banco local
npm run dev                    # http://localhost:5173
```

Rodar local uma vez economiza um ciclo de CI: erros de configuração de arquivo aparecem aqui.

32. Migrações do banco

32.1 As três formas

```
npm run db:migrate             # LOCAL
npm run db:migrate:staging     # STAGING (banco dedicado)
npm run db:migrate:prod -- --yes # PRODUÇÃO (o --yes é obrigatório)
```

O runner (`scripts/migrate.ts`) é **incremental**: lê a tabela de controle `_migrations_aplicadas` e executa só o que falta. Rodar de novo é barato e idempotente — é por isso que o CI o executa em todo deploy.

⚠ Só a produção remota exige `--yes`. Staging tem banco próprio, e local é descartável. A salvaguarda existe porque `db:migrate:staging` e `db:migrate:prod` diferem por um Tab no autocomplete.

32.2 O caso especial: banco que já está migrado

Se você está adotando este controle num banco **que já tem o schema aplicado** por outro caminho:

```
npm run db:migrate:prod -- --yes --baseline
```

Isso marca **todas** as migrações atuais como aplicadas **sem executá-las**. Use uma única vez. Depois disso, só as novas rodam.

32.3 O que algumas migrações SEMEIAM

Nem toda migração só cria tabela. Vale saber o que esperar:

Migração	O que ela grava
0048	Cria a operação GISE e faz backfill de todas as escalas existentes para ela
0050	Insere a OPERAÇÃO CRAJUBAR , copia os formulários do GISE e cria cinco indicadores
0051	Acrescenta configuração por operação — tudo nasce NULL ("herda o padrão"), nenhum PDF muda
0052	Converte um indicador da CRAJUBAR para meta de cobertura (muda as chaves de resposta)
0072	Cria a matriz de municípios e distâncias — ~ 440 KB de dados versionados (184 municípios, 16.836 pares). Executa em menos de um segundo, sem passo manual
0074	Semeia as três regiões metropolitanas do Ceará

Num banco **novo**, tudo isso simplesmente nasce pronto. Num banco **em uso**, confira depois do deploy que `/gise/operacoes` mostra as operações e que as escalas antigas exibem o selo correto.

32.4 Ordem segura

✅ **Sempre staging antes de produção.** Migração não tem *down* automático. Reverter é (a) `time-travel restore` para o instante anterior, ou (b) escrever uma migração corretiva — preferível para mudanças pequenas.

33. O primeiro deploy

33.1 Pelo GitHub Actions (recomendado)

1. Confirme os secrets do capítulo 30.
2. Faça push para `staging` primeiro. Acompanhe a aba **Actions**.
3. Com o staging verde e testado, abra o PR de `staging` para `main` e mergeie.

33.2 Manual (fallback)

```
npm run db:migrate:prod -- --yes
npm ci
npm run build
wrangler pages deploy .svelte-kit/cloudflare --project-name=escalas
```

⚠ Sempre migre antes de publicar código novo. As migrações do projeto são retrocompatíveis (*expand/contract*), então o código antigo convive por instantes com o schema novo — o contrário não é verdade.

33.3 Se o deploy falhar

Erro	Causa provável
403 no <code>pages deploy</code>	Token sem Cloudflare Pages — Edit
403 no passo de migração	Token sem D1 — Edit
"project not found"	Projeto Pages não criado, ou nome diferente de <code>escalas</code>
Erro de binding D1	<code>database_id</code> não trocado no <code>wrangler.toml</code>
Falha no preview, produção ok	<code>database_id</code> do staging ainda é placeholder (é <i>fail-safe</i> proposital)

34. Smoke tests pós-deploy

Faça os seis, na ordem. Levam menos de dez minutos.

1. Liveness

```
curl -s https://<seu-dominio>/api/health
# {"status":"ok"}
```

2. Segredos de proteção

```
curl -s "https://<seu-dominio>/api/health?detail=<HEALTH_DETAIL_TOKEN>" | jq
'.protecoesAusentes, .checks'
```

`protecoesAusentes` precisa vir **vazio** (`[]`). Qualquer nome ali é uma variável faltando — volte à Parte II antes de seguir.

3. Login — entre com a conta de bootstrap (capítulo 35). Se o `SUPER_ADMIN_EMAIL` estiver configurado, o 2FA precisa chegar. **Se o e-mail não chegar, pare aqui:** nada mais funciona sem e-mail.

4. Cadeia de auditoria — `/auditoria` → **Verificar integridade** → precisa dizer `HMAC-SHA256`.

5. Um documento ponta a ponta — crie uma escala de teste, gere o PDF, assine e valide em `/validar` com o código impresso.

6. Logs — confira o painel (Workers & Pages → `escalas` → Logs) e o Sentry: nenhum erro inesperado.

35. As primeiras contas

35.1 Entrar como Super Admin

Com `SUPER_ADMIN_LOGIN` e `SUPER_ADMIN_SENHA` configurados, entre pela aba **Administrador** do `/login`. Se `SUPER_ADMIN_EMAIL` estiver definido, o 2FA será exigido — como deve ser.

 Todo login por bootstrap gera evento de auditoria e um `warning` no Sentry. Isso é intencional: a conta root não deve ser o login do dia a dia.

35.2 A ordem de povoamento

1. Unidades → `/unidades` (Super Admin)
2. Policiais → `/policiais/upload` (CSV) ou sincronização pela planilha
3. Papéis → `/policiais/[id]` — promover admins de seccional/unidade
4. Operações → `/gise/operacoes` (Admin Geral)
5. Valores de custo → `/config-custos` (Super Admin)
6. Política de assinatura → `/conf-ass` (Super Admin)

 **Unidade não se exclui, só se desativa** — e isso vale desde o primeiro cadastro. O nome da unidade é a chave que amarra lotação, escala e cabeçalho de documento; não existe ação de excluir na interface e o banco recusa o `DELETE`. Cadastre com o nome oficial, correto, desde o começo.

35.3 Quem pode o quê

Capacidade	Super Admin	Admin Geral
Promover admins, gerenciar policiais e unidades	✓	✗
Configurar política de assinatura	✓	✗
Consoles de auditoria	✓	✗
Baixar o forense pelo portal <code>/validar</code>	✓	✗
Operar escalas, GISE, LGPD em todas as unidades	✓	✓
Decidir solicitações de cadastro e atos de RH	✓	✓

Os administradores **de banco** nascem só dos bootstraps por env. Os administradores **operacionais** (seccional/unidade) são policiais promovidos, e só o Super Admin promove.

35.4 Desligue o bootstrap do Admin Geral

Depois que o Admin Geral tiver e-mail configurado, o login por env é **bloqueado automaticamente** (o sistema registra um erro explícito no log pedindo a remoção).  Remova `ADMIN_GERAL_LOGIN` e `ADMIN_GERAL_SENHA` do ambiente.

O `SUPER_ADMIN_*` **permanece** — é o break-glass, e não deve ser removido. Mantenha-o lacrado: senha em hash `pbkdf2v2` e `SUPER_ADMIN_EMAIL` definido.

36. Configuração inicial do sistema

36.1 Política de assinatura (`/conf-ass` , Super Admin)

Flag	O que faz	Cuidado
<code>exigir_foto_assinatura</code>	Selfie com prova de vivacidade	Sem câmera, o servidor não assina em tela
<code>exigir_gps_assinatura</code>	Coordenada no ato	Aceita ausência declarada , com motivo de lista fechada
<code>exigir_codigo_email_assinatura</code>	2FA no ato de assinar	Depende do e-mail funcionando
<code>restringir_smartphone</code>	Recusa assinatura avançada em desktop	Aplicada no servidor , não só na tela
<code>exigir_passkey_assinatura</code>	Exige chave WebAuthn do celular	Ver abaixo

⚠ Antes de ligar `exigir_passkey_assinatura` :

1. `APP_ORIGIN` definida e **igual ao domínio final** — a credencial fica presa ao domínio.
2. Adesão medida: **quem não registrou a chave em `/perfil` não assina** (o fluxo antigo passa a responder 403, de propósito — reforço contornável não é reforço).
3. Aparelhos: iOS 16+ ou Android 9+, **com bloqueio de tela configurado**.

💡 As flags são cacheadas por 5 minutos em todos os pontos de presença; ao alterar, o sistema invalida o cache automaticamente. Confirme que a mudança reflete em ≤ 5 min.

36.2 Valores de custo (`/config-custos` , Super Admin)

⚠ A tabela de valores precisa existir **ANTES do primeiro plano operacional**. A tabela nasce vazia e o módulo **não** recusa por isso: o plano é criado, o editor abre — e o Anexo II do PDF sai **zerado**.

Preencha: hora extra por faixa de cargo/classe (DPC 1ª/2ª, DPC 3ª/especial, OIP A/B, OIP C/D), as duas diárias e a distância mínima para diária (padrão 100 km).

A tabela é **append-only e versionada**: cada gravação é uma versão nova, e cada plano guarda a versão que usou. Reajuste posterior **não** reescreve documento já emitido.

36.3 Modelos de reconhecimento facial

Os arquivos do `face-api` estão versionados em `static/face-api/` e são servidos pela própria CDN do Pages. **Nada a configurar**.

💡 Ao atualizar a biblioteca `@vladmandic/face-api`, copie os arquivos novos do `node_modules` para `static/face-api/` no mesmo PR — senão a versão da lib em runtime fica dessincronizada do modelo servido.

37. Go-live: o reset de senhas

Este é o passo irreversível. Leia inteiro antes de executar.

37.1 Pré-requisitos absolutos

1. **E-mail funcionando em produção**, testado de ponta a ponta.
2. **Uma conta de teste** já validada no ciclo completo: primeiro acesso → senha → verificação de e-mail → 2FA → login → logout → login de novo.
3. Comunicação pronta para os servidores explicando como fazer o primeiro acesso.

37.2 O comando

```
CONFIRMO_PRODUCAO=escalas-db \
  npm run users:clear-passwords-non-admins:prod -- --yes
```

Ele zera a senha e marca `primeiro_acesso = 1` para todos os policiais, **preservando os administradores**.

⚠️ **Duas confirmações, e não é excesso de zelo.** O `--yes` não vem embutido no `npm run`, e contra produção a variável `CONFIRMO_PRODUCAO` precisa conter o **nome do banco**. O comando local e o de produção diferem por um sufixo `:prod`, e este **não tem desfazer**: o `UPDATE` sobrescreve o `updated_at` de todo mundo — depois nem se distingue quem já estava sem senha de quem o comando atingiu.

37.3 Contas em formato de hash antigo

Hashes anteriores ao PBKDF2 (SHA-256 sem salt) **não são mais aceitos**. Contas nesse estado não autenticam por senha e precisam passar pelo reset acima.

38. Checklist de go-live

Imprima e marque.

Infraestrutura

- ☐ D1 de produção e staging criados, `database_id` trocados no `wrangler.toml`
- ☐ R2 `escalas-docs`, `escalas-docs-staging` e `escalas-backups` criados
- ☐ Nenhum bucket com acesso público
- ☐ Lifecycle e retention lock no bucket de backup
- ☐ Projeto Pages `escalas` criado como Direct Upload (não conectado ao Git)
- ☐ Domínio próprio ativo com TLS

Variáveis (Production e Preview)

- ☐ SYNC_TOKEN (≥ 32 caracteres)
- ☐ RESET_TOKEN **diferente** do SYNC_TOKEN (ou vazio, desabilitando o reset)
- ☐ E-mail: binding EMAIL **ou** RESEND_API_KEY + RESEND_FROM_EMAIL
- ☐ PASSWORD_PEPPER definido **e no cofre**
- ☐ CPF_ENCRYPTION_KEY e CPF_INDEX_KEY (distintas entre si)
- ☐ AUDIT_CHAIN_KEY e AUDIT_IP_ENCRYPTION_KEY
- ☐ RATE_LIMIT_IP_SALT
- ☐ APP_ORIGIN no domínio canônico
- ☐ HEALTH_DETAIL_TOKEN
- ☐ SUPER_ADMIN_EMAIL e ADMIN_GERAL_EMAIL definidos (2FA no bootstrap)
- ☐ Senhas de bootstrap em hash pbkdf2v2: , não em texto
- ☐ WEBHOOK_REPLAY_ENFORCE=1
- ☐ WEBHOOK_ALLOW_PAPIL_CHANGES **vazia**
- ☐ EMBED_PADES_LT_DSS **vazia**
- ☐ ICP_BRASIL_TRUST_STORE_REQUIRED=1 (com os PEMs populados)
- ☐ TSA_URL apontando para ACT credenciada, se for exigir carimbo qualificado
- ☐ EXIGIR_TSA_QUALIFICADA=1 **somente depois** de trocar a TSA_URL
- ☐ SELO_INSTITUCIONAL_PEM (se usar assinatura avançada)
- ☐ SENTRY_DSN e PUBLIC_SENTRY_DSN

GitHub

- ☐ CLOUDFLARE_API_TOKEN e CLOUDFLARE_ACCOUNT_ID
- ☐ APP_BASE_URL (produção, sem barra final) e SYNC_TOKEN (mesmo valor do Cloudflare)
- ☐ CLOUDFLARE_BACKUP_API_TOKEN e BACKUP_AGE_PUBLIC_KEY
- ☐ Environments production e staging criados, com proteção em production
- ☐ Proteção de branch em main exigindo o check test
- ☐ Dependabot alerts ligados
- ☐ Backup disparado manualmente **e restaurado num banco descartável**

Aplicação

- ☐ Migrações aplicadas em staging e produção
- ☐ /api/health?detail= com protecoesAusentes vazio
- ☐ /auditoria → integridade em HMAC-SHA256
- ☐ Login real validado (login → logout → login de novo)
- ☐ Um documento assinado e validado em /validar
- ☐ Unidades e policiais cadastrados
- ☐ Valores de custo preenchidos em /config-custos
- ☐ Política de assinatura definida em /conf-ass

- ☐ Monitor externo apontado para `retencao.atrasada`
- ☐ Reset de senhas executado e comunicado

Parte V — Operação

39. A rotina

Diária (automática, você só confere)

Hora (UTC)	O quê	Onde conferir
04:17	Limpeza de retenção (LGPD)	Aba Actions; <code>retencao.atrasada</code> no health
05:43	Backup cifrado do D1	Aba Actions; objeto novo no bucket

Semanal

- Revisar os PRs do Dependabot (mergeie só com o CI verde).
- Olhar o Sentry: erros novos, e especialmente qualquer `[CADES][CONFIG]`.
- Conferir `pendenciasRestantes` da limpeza — **zero é o normal**.

Mensal

- Revisar e mergeiar o PR do trust store ICP-Brasil, se houver.
- Conferir o crescimento do D1 e do R2 contra a cota.

Trimestral

- **Teste de restauração do backup** num banco descartável.
- Revisar quem tem papel administrativo (`/policiais`) e quem tem acesso ao painel Cloudflare e ao repositório.
- Conferir que os segredos do cofre continuam legíveis e conferem com o ambiente.

40. Rollback

O que deu errado	Como reverter
Código	Painel → Pages → Deployments → no último deployment bom, Rollback to this deployment . Instantâneo, sem rebuild, não afeta D1/R2
Migração	<code>wrangler d1 time-travel restore</code> para o instante anterior, ou uma migração corretiva nova (preferível para mudanças pequenas)
Dados apagados	Time Travel (~30 dias) ou o backup diário cifrado
Documento assinado perdido no R2	Não há PITR nativo no R2. O <code>arquivo_hash</code> no D1 permite detectar objetos ausentes; a recuperação depende de versionamento/lock configurados no bucket

⚠ Antes de qualquer `time-travel restore`, **faça um export**. O restore substitui o estado atual: tudo que foi gravado depois do ponto escolhido se perde.

💡 Cópias de conferência (`conferencia/<hash>.pdf`) são regeneráveis — perdê-las é inócuo. O que não se recupera é o **blob assinado**.

41. Rotação de segredos

41.1 O que NUNCA se rotaciona

Segredo	Por quê
<code>PASSWORD_PEPPER</code>	Invalida todos os hashes <code>v3</code> — a base inteira redefine senha
<code>CPF_ENCRYPTION_KEY</code>	Os CPFs cifrados viram lixo
<code>CPF_INDEX_KEY</code>	O índice cego para de casar
<code>AUDIT_CHAIN_KEY</code>	As linhas antigas da trilha ficam inverificáveis
<code>SELO_INSTITUCIONAL_PEM</code>	Muda a identidade do selo (e você nunca regenera a mesma chave)

Se a rotação for inevitável (comprometimento), ela deixa de ser troca de variável e vira **projeto**: plano de migração de dado, janela de manutenção e comunicação.

41.2 O que se rotaciona, com cuidado

Segredo	Onde trocar junto
<code>SYNC_TOKEN</code>	Cloudflare (Production) + secret do GitHub + Apps Script da planilha
<code>RESET_TOKEN</code>	Cloudflare + Apps Script
<code>GISE_BASE_EQUIPE_SECRET</code>	Cloudflare + Script Properties da planilha
<code>CLOUDFLARE_API_TOKEN</code>	Painel Cloudflare + secret do GitHub

⚠ O cron de limpeza é **o primeiro a quebrar** quando só um lado do `SYNC_TOKEN` é atualizado — e quebra em silêncio até alguém abrir a aba Actions. É para isso que serve o monitor de `retencao.atrasada`.

41.3 O que se rotaciona sem drama

`RATE_LIMIT_IP_SALT` (só reseta as janelas de rate-limit em andamento), `HEALTH_DETAIL_TOKEN`, `RESEND_API_KEY`, `SENTRY_DSN`, `TSA_PASSWORD`.

42. Solução de problemas por sintoma

Ninguém consegue logar

Sintoma	Causa provável	Verificação
Código 2FA não chega	E-mail não configurado ou remetente inválido	Logs do Worker: <code>[email/cloudflare]</code> e a resposta do Resend
Envio pela Cloudflare falha sempre	Domínio do <code>CF_FROM</code> não é seu	<code>src/lib/server/email.ts</code> , constante <code>CF_FROM</code>
Login com senha correta é recusado	Hash <code>v3</code> com pepper trocado/ausente	<code>/api/health?detail=</code> → <code>checks.senhaPepper</code>
"Muitas tentativas" para a unidade inteira	<code>RATE_LIMIT_IP_SALT</code> ausente (bloqueio por /24)	<code>checks.rateLimitIpSalt</code> e <code>scripts/diagnostico-salt-rate-limit.sql</code> , que diz se o salt chegou às chaves já gravadas
Sessão cai logo após entrar	Relógio fora de sincronia (o D1 usa UTC)	NTP da máquina que consulta

 A sessão dura **1 hora de inatividade**, com renovação a cada ação real. Quem deixa a aba aberta e volta depois de uma hora encontra o login — é o controle funcionando; vale avisar a corporação.

Assinatura falhando

Sintoma	Causa provável
HTTP 422 em toda assinatura qualificada	<code>EXIGIR_TSA_QUALIFICADA=1</code> com <code>TSA_URL</code> não-ICP (a DigiCert do default). Procure <code>[CADES]</code> <code>[CONFIG]</code> no log
"Cadeia indisponível"	Trust store vazio; popule os PEMs
Assinatura aceita, rótulo fraco	<code>TSA_URL</code> não é ACT credenciada — carimbo vira <code>tsa_externa</code>
Passkey não funciona depois de trocar o domínio	Credencial presa ao domínio antigo (<code>APP_ORIGIN</code>). Não tem conserto: exige recadastro
Adobe marca assinatura como inválida	<code>EMBED_PADES_LT_DSS</code> ligada — deixe vazia
Preparar responde 200 e finalizar responde 500	Controle de migrações misturado (<code>d1_migrations</code> × <code>_migrations_aplicadas</code>) — capítulo 6.3

Automação parada

Sintoma	Causa
Cron de limpeza com 401	<code>SYNC_TOKEN</code> diferente do Cloudflare; <code>APP_BASE_URL</code> apontando para preview; token com menos de 32 caracteres
Sincronização da planilha com 401	<code>WEBHOOK_REPLAY_ENFORCE=1</code> com Apps Script não republicado (sem os headers)
<code>retencao.atrasada: true</code>	Workflow desabilitado, secret rotacionado de um lado só, ou cota de Actions esgotada
Backup sem objeto novo	Secrets do backup ausentes, ou token sem permissão de export
PR do trust store nunca abre	Permissão de workflow sem "create pull requests"

Ambiente errado

Sintoma	Causa
Preview falha ao ligar o banco	<code>database_id</code> de staging ainda é placeholder — é proposital
Staging escrevendo em produção	Bindings de preview não aplicados; confira no painel (Settings → bindings, escopo Preview)
Variável salva e nada mudou	Falta um novo deploy; ou foi cadastrada como variável de build em vez de runtime

43. Palavra final

Uma instalação deste sistema termina quando três coisas são verdade ao mesmo tempo:

1. `/api/health?detail=` responde com `protecoesAusentes` **vazio**;
2. `/auditoria` → Verificar integridade responde **HMAC-SHA256** ;
3. um backup já foi **restaurado** com sucesso num banco descartável.

Nenhuma das três aparece na tela para o usuário final, e é justamente por isso que precisam estar numa lista. O sistema sobe e funciona sem elas — só não protege nada.

Apêndices

Apêndice A — Modelo de inventário de variáveis

Copie esta lista para o cofre da instituição e preencha à medida que configurar. A coluna "Onde" indica o escopo no Cloudflare Pages.

```
# — Obrigatórias —————
SYNC_TOKEN=                # Production + Preview 🔒
APP_ORIGIN=                 # Production (e Preview com a URL de preview)
RESEND_API_KEY=            # Production + Preview 🔒
RESEND_FROM_EMAIL=        # Production + Preview

# — Proteções que degradam em silêncio —————
PASSWORD_PEPPER=          # Production (+ valor de teste no Preview) 🔒 NUNCA
ROTACIONAR
CPF_ENCRYPTION_KEY=       # Production + Preview 🔒 NUNCA ROTACIONAR
CPF_INDEX_KEY=            # Production + Preview 🔒 NUNCA ROTACIONAR (distinta da
acima)
AUDIT_CHAIN_KEY=          # Production + Preview 🔒 NUNCA ROTACIONAR
AUDIT_IP_ENCRYPTION_KEY=  # Production + Preview 🔒
RATE_LIMIT_IP_SALT=       # Production + Preview 🔒

# — Contas de bootstrap —————
SUPER_ADMIN_LOGIN=        # Production
SUPER_ADMIN_SENHA=        # Production 🔒 (use hash pbkdf2v2 – apêndice D)
SUPER_ADMIN_EMAIL=        # Production
ADMIN_GERAL_LOGIN=        # Production (remover após o setup)
ADMIN_GERAL_SENHA=        # Production 🔒 (remover após o setup)
ADMIN_GERAL_EMAIL=        # Production

# — Webhooks —————
RESET_TOKEN=              # Production 🔒 (DIFERENTE do SYNC_TOKEN)
WEBHOOK_REPLAY_ENFORCE=1  # Production
WEBHOOK_ALLOW_PAPEL_CHANGES= # deixar VAZIA

# — Assinatura digital —————
ICP_BRASIL_TRUST_STORE_REQUIRED=1 # Production
TSA_URL=                  # Production (ACT credenciada)
TSA_USERNAME=             # se a ACT exigir 🔒
TSA_PASSWORD=             # se a ACT exigir 🔒
EXIGIR_TSA_QUALIFICADA=   # só DEPOIS de trocar a TSA_URL
SELO_INSTITUCIONAL_PEM=   # Production 🔒 NUNCA REGENERÁVEL
```

```

MBED_PADES_LT_DSS=                # deixar VAZIA
PA_AD_RB_HASH_HEX=                # deixar vazia (valor oficial no código)

# — Observabilidade e ajuste —————
SENTRY_DSN=                        # Production + Preview
SENTRY_ENVIRONMENT=production      # Production ('staging' no Preview)
SENTRY_TRACES_SAMPLE_RATE=0.1
PUBLIC_SENTRY_DSN=                 # Production + Preview
PUBLIC_SENTRY_ENVIRONMENT=production
HEALTH_DETAIL_TOKEN=               # Production + Preview 🗝️
SESSION_CACHE_TTL_SECONDS=         # opcional (default 60)

# — E-mail via API da Cloudflare (se não usar o binding) —————
CLOUDFLARE_API_TOKEN=              # Production 🗝️
CLOUDFLARE_ACCOUNT_ID=             # Production

# — Integração com a planilha —————
GISE_BASE_EQUIPE_WEBHOOK_URL=      # Production
GISE_BASE_EQUIPE_SECRET=           # Production 🗝️

```

Apêndice B — Comandos de referência

```

# — Cloudflare —————
wrangler login
wrangler whoami                    # Account ID

wrangler d1 create escalas-db
wrangler d1 create escalas-db-staging
wrangler d1 export escalas-db --remote --output=backup-$(date +%F).sql
wrangler d1 time-travel info escalas-db --remote
wrangler d1 time-travel restore escalas-db --remote --timestamp="AAAA-MM-DDTHH:MM:SSZ"
wrangler d1 execute escalas-db --remote --command "SELECT * FROM _migrations_aplicadas ORDER BY id DESC LIMIT 10"

wrangler r2 bucket create escalas-docs
wrangler r2 object get escalas-backups/d1/diario/<arquivo> --file=<destino> --remote

wrangler pages project create escalas --production-branch=main
wrangler pages deploy .velte-kit/cloudflare --project-name=escalas
wrangler pages secret put <NOME> --project-name=escalas
wrangler pages secret put <NOME> --project-name=escalas --env=preview
wrangler pages secret list --project-name=escalas

# — Projeto —————

```

```

npm ci
npm run build
npm run db:migrate                # local
npm run db:migrate:staging
npm run db:migrate:prod -- --yes
npm run db:migrate:prod -- --yes --baseline    # adoção única

CONFIRMO_PRODUCAO=escalas-db npm run users:clear-passwords-non-admins:prod -- --yes

node scripts/gerar-selo-institucional.mjs "CN do selo" "Organizacao"

# — Segredos —————
openssl rand -hex 32
age-keygen -o backup-key.txt

# — Conferência —————
curl -s https://<dominio>/api/health
curl -s "https://<dominio>/api/health?detail=<TOKEN>" | jq

# O salt do rate-limit chegou às chaves gravadas? (cruze com o health acima –
# a matriz das quatro combinações está no cabeçalho do arquivo)
npx wrangler d1 execute escalas-db --remote \
  --file=scripts/diagnostico-salt-rate-limit.sql

```

Apêndice C — Permissões dos tokens de API

Token de deploy — `CLOUDFLARE_API_TOKEN`

Escopo	Permissão	Nível
Conta	Cloudflare Pages	Edit
Conta	D1	Edit
Conta	Account Settings	Read (só se o Wrangler reclamar de resolução de conta)

Token de backup — `CLOUDFLARE_BACKUP_API_TOKEN`

Escopo	Permissão	Nível
Conta	D1	Read (promova para Edit se o <code>export</code> devolver 403)
Conta	Workers R2 Storage	Edit , restrito ao bucket <code>escalas-backups</code>

Token de e-mail — se usar o caminho REST da Cloudflare

Escopo	Permissão	Nível
Conta	Email Sending	Edit

Apêndice D — Gerar o hash da senha de bootstrap

O script vive em `scripts/hash-senha.mjs`. Ele produz o formato que o sistema aceita (`pbkdf2v2:<iterações>:<salt hex>:<hash hex>` — PBKDF2-HMAC-SHA256, 100 000 iterações, salt de 16 bytes, saída de 32 bytes), e `src/lib/crypto/__tests__/hash-senha-script.test.ts` executa o arquivo de verdade e confere o resultado contra o `verificarSenha` do login, para que os parâmetros não divirjam em silêncio.

Conteúdo, para referência:

```
// Gera hash pbkdf2v2 para SUPER_ADMIN_SENHA / ADMIN_GERAL_SENHA.
// Uso: HASH_PASSWORD='SuaSenhaForte' node hash-senha.mjs
const senha = process.env.HASH_PASSWORD;
if (!senha) {
  console.error("uso: HASH_PASSWORD='senha' node hash-senha.mjs");
  process.exit(1);
}
const ITER = 100000;
const salt = crypto.getRandomValues(new Uint8Array(16));
const km = await crypto.subtle.importKey(
  'raw', new TextEncoder().encode(senha), 'PBKDF2', false, ['deriveBits']
);
const bits = await crypto.subtle.deriveBits(
  { name: 'PBKDF2', salt, iterations: ITER, hash: 'SHA-256' }, km, 256
);
const hex = (b) => [...new Uint8Array(b)].map((x) => x.toString(16).padStart(2, '0')).join('');
console.log(`pbkdf2v2:${ITER}:${hex(salt)}:${hex(bits)}`);
```

```
HASH_PASSWORD='SuaSenhaForte' node scripts/hash-senha.mjs
# pbkdf2v2:100000:8726693d34de607dffe209dcfa7cc785:08cd5b47...
```

Cole a saída inteira (com o prefixo `pbkdf2v2:`) na variável `SUPER_ADMIN_SENHA`.



Por que v2 e não v3: o hash do bootstrap é conferido **sem** o pepper, de propósito — assim a conta root continua entrando mesmo se o `PASSWORD_PEPPER` for perdido. É o break-glass funcionando como projetado.



Guarde a **senha em claro** no cofre também: o hash não é reversível, e sem a senha você não entra.

Apêndice E — Glossário de implantação

Termo	Significado
Binding	Recurso da Cloudflare (banco, bucket, e-mail) ligado ao Worker por um nome
D1	Banco SQLite gerenciado da Cloudflare
R2	Armazenamento de objetos da Cloudflare, compatível com S3
Pages	Hospedagem da Cloudflare onde a aplicação roda como Worker
Direct Upload	Projeto Pages publicado por CLI/CI, sem conexão com o Git
Preview	O ambiente não-produção do Pages — aqui, o staging
Wrangler	A CLI da Cloudflare
Fail-closed	Na dúvida, recusa (ex.: 2FA sem e-mail não cria sessão)
Fail-open	Na falta, segue sem a proteção (ex.: CPF sem chave grava em texto)
Load-bearing	Segredo cuja troca destrói dado já gravado
Expand/contract	Migração retrocompatível: o código antigo convive com o schema novo
Time Travel	Recuperação do D1 para qualquer ponto dos últimos ~30 dias
age	Ferramenta de cifragem usada no backup
RP ID	O domínio ao qual uma credencial WebAuthn fica presa
ACT	Autoridade de Carimbo do Tempo credenciada ICP-Brasil
TSA	Time-Stamp Authority (RFC 3161)
Break-glass	Credencial de emergência, usada só quando o caminho normal falhou